⚙️ Jenkins Architecture & Internals — Complete Interview Mastery Guide

Category 2 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic follows the full teaching structure: Simple Definition → Why it exists → How it works internally → Key concepts → Short interview answer → Deep dive → Real-world production example → Interview Q&A → Tricky gotchas → Connections to other topics.
>
> ⚠ = Frequently misunderstood or heavily tested. Give these extra attention.
>
> This is the **engine room category** — senior interviewers probe Jenkins architecture deeply because it reveals whether you've actually operated Jenkins at scale or just used it as a black box. The difference between someone who's administered Jenkins and someone who's only consumed it is obvious in 3 questions.

📜 TABLE OF CONTENTS

1. Topic 2.1 — Jenkins Architecture: Master/Controller, Agents, Executors
2. Topic 2.2 — Jenkins Installation, System Configuration & Global Tools
3. Topic 2.3 — Jenkins Controller Internals: Job Queue, Build Queue, Thread Model
4. Topic 2.4 — Agent Types: Static, Dynamic, Docker, Kubernetes, SSH, JNLP
5. Topic 2.5 — Jenkins Workspace: How It Works, Cleanup, Shared Workspaces
6. Topic 2.6 — Jenkins Home Directory Structure
7. Topic 2.7 — Jenkins HA and Disaster Recovery ⚠
8. Topic 2.8 — Jenkins at Scale: Controller Bottlenecks, Agent Pools, Load Distribution
9. Category 2 Summary & Self-Quiz

Topic 2.1 — Jenkins Architecture: Master/Controller, Agents, Executors

🟢 Beginner | The Foundation

📌 What It Is — In Simple Terms

Jenkins uses a **distributed architecture** composed of three tiers:

1. **Controller** (historically called "Master") — the central brain: schedules jobs, stores configuration, serves the UI, manages plugins, and orchestrates agents

2. **Agent** (historically called "Slave" or "Node") — a machine or container that actually executes build steps on behalf of the Controller
3. **Executor** — a single thread slot on an agent that runs one build at a time

```
┌──────────────────────────────────────────────────────────────────────┐
│                        JENKINS CONTROLLER                            │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│  │  Web UI      │  │  Job         │  │  Plugin      │  │  Build       │  │
│  │  (Jetty)     │  │  Scheduler   │  │  Manager     │  │  Queue       │  │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │                  JENKINS_HOME (/var/jenkins_home)                │  │
│  │  jobs/  builds/  plugins/  config.xml  nodes/  workspace/        │  │
│  └──────────────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
                  │  Agent connection (SSH / JNLP / WebSocket)
        ┌─────────────────────────────────┐
        │                                 │
┌───────▼──────────────┐          ┌───────▼──────────────┐
│   AGENT: linux-1     │          │   AGENT: docker-1    │
│                      │          │                      │
│  ┌────────────────┐  │          │  ┌────────────────┐  │
│  │ Executor 1     │  │          │  │ Executor 1     │  │
│  │ (Build #45)    │  │          │  │ (Build #46)    │  │
│  └────────────────┘  │          │  └────────────────┘  │
│                      │          │                      │
│  ┌────────────────┐  │          │  ┌────────────────┐  │
│  │ Executor 2     │  │          │  │ Executor 2     │  │
│  │ (idle)         │  │          │  │ (idle)         │  │
│  └────────────────┘  │          │  └────────────────┘  │
│                      │          │                      │
│  Labels: linux,      │          │  Labels: docker,     │
│  java, maven         │          │  node, python        │
└──────────────────────┘          └──────────────────────┘
```

🔍 Why This Architecture Exists — The Problem It Solves

Before distributed builds: CI servers ran everything on one machine. Problems:

- Build capacity limited to one machine's CPUs
- Different projects need different OS, tools, JDK versions — impossible on one machine
- One slow build blocks all other builds
- Security risk: all builds share the same filesystem and OS user

The distributed architecture solves:

Problem	Solution
Limited compute	Add more agents — horizontal scale

Problem	Solution
Different environments needed	Agents with different labels (java11, python3, docker)
Build isolation	Each build gets its own workspace on an agent
Security	Controller doesn't run user code; agents can be locked down
Specialization	GPU agents for ML, Windows agents for .NET, Mac agents for iOS

⚙️ How It Works Internally — The Full Communication Flow

```
Step 1: Developer pushes code → webhook fires to Jenkins Controller

Step 2: Controller receives webhook
  → Parses the payload (which repo, which branch)
  → Finds matching job (Multi-Branch Pipeline scan)
  → Creates a build item and adds it to the BUILD QUEUE

Step 3: Build Queue management
  → Controller evaluates label expressions for the job
    (e.g., agent { label 'linux && docker' })
  → Waits for an agent matching labels to have a free executor

Step 4: Agent selection
  → Controller picks the least loaded matching agent
  → Instructs agent: "start executor, run this build"

Step 5: Build execution on Agent
  → Agent receives pipeline steps from Controller
  → Agent executes steps locally (git checkout, mvn build, docker build)
  → Agent streams logs back to Controller in real-time
  → Agent stores workspace files locally

Step 6: Build completion
  → Agent reports result (SUCCESS/FAILURE/UNSTABLE) to Controller
  → Controller archives artifacts (copies from agent to JENKINS_HOME)
  → Controller records build history
  → Controller sends notifications (Slack, email)
  → Agent workspace may be cleaned or retained
```

🔑 Key Components

Component	Role	Where It Runs
Controller	Orchestration, UI, job definitions, plugin management, build history	Dedicated server/pod
Agent	Executes pipeline steps — the actual build worker	Any machine, VM, container, cloud instance

Component	Role	Where It Runs
Executor	A single build slot on an agent — one executor = one concurrent build	Within an agent
Label	Tag applied to agents — jobs use label expressions to target agents	Defined on agent
Build Queue	List of builds waiting for a free executor	On Controller, in memory
Workspace	Working directory on agent where code is checked out and builds run	On Agent's filesystem
Channel	The communication channel between Controller and Agent (Remoting library)	Network connection

---

💬 Short Crisp Interview Answer

> *"Jenkins uses a Controller-Agent architecture. The Controller is the central server — it handles the Web UI, job scheduling, plugin management, build queue, and stores all configuration and history in JENKINS_HOME. The Controller itself doesn't run user build code. Agents are the workers — separate machines, VMs, containers, or pods that the Controller connects to and delegates build execution. Each agent has a configurable number of Executors — each executor is one concurrent build slot. Agents are labeled, and pipeline jobs specify label expressions to target the right type of agent. This architecture separates orchestration (Controller) from execution (Agent), enabling horizontal scaling by adding more agents and supporting heterogeneous build environments through labels."*

🔬 Deep Dive Answer

The Remoting Library — How Controller and Agent Actually Communicate:

Jenkins uses its own **Remoting** library (based on Java serialization and NIO) for Controller-Agent communication. This is important because it's the source of several security and performance characteristics:

```
Controller ←── Remoting Protocol ──→ Agent

The channel carries:
  1. Commands: "checkout this git repo", "run this shell command"
  2. Class loader: Agent receives Java bytecode (pipeline steps) from
Controller
  3. Log streaming: Agent sends stdout/stderr back to Controller in real-
time
  4. File transfer: Artifacts copied from Agent to Controller

Protocol options:
  - SSH: Controller SSH's into agent, starts agent.jar (most common for
```

```
static agents)
  - JNLP/TCP: Agent initiates outbound TCP connection to Controller (good
for agents behind NAT)
  - WebSocket: Agent connects via WebSocket (good for environments
blocking raw TCP)
  - Inbound agent: Agent polls Controller (cloud/k8s agents)
```

The Remoting serialization security issue:

Jenkins Remoting historically used Java serialization, which is notoriously vulnerable. The 2015-2016 "Jenkins Remoting" vulnerabilities (CVE-2015-8103, etc.) allowed unauthenticated RCE via the JNLP port. Modern Jenkins mitigates this by:

- Requiring authentication for agent connections
- Supporting Remoting over WebSocket (avoids exposing TCP 50000)
- Using TLS for all agent communications
- Running agents as least-privilege users

Why the Controller should NEVER run builds:

```
# In production: ALWAYS set Controller executors to 0
# Manage Jenkins → Nodes → Built-In Node → Number of executors → 0

Reasons:
1. Security: pipeline scripts (Groovy) run in Jenkins JVM
   If Controller has executors, malicious pipeline code runs in Controller
process
   With 0 executors, Controller never executes user code

2. Stability: Controller JVM under build load competes with
   queue management, UI serving, plugin operations
   → Controller becomes slow/unresponsive under build load

3. Isolation: build artifacts, workspaces, temp files stay on agents
   → JENKINS_HOME stays clean, predictable size

4. Resource contention: builds consume CPU/memory
   → Controller should have predictable resource usage for stability
```

Agent Labels — The Routing System:

```
// Label expressions control which agent runs a job

// Simple label:
agent { label 'linux' }

// Boolean AND — must match BOTH labels:
agent { label 'linux && docker' }
```

```
// Boolean OR — either label matches:
agent { label 'ubuntu || debian' }

// Complex expression:
agent { label '(linux && java11) || (linux && java17)' }

// Any available agent:
agent any

// Specific named agent (avoid in production — creates hard dependency):
agent { node { label 'agent-hostname-1' } }

// No agent (for declarative pipeline with stages on different agents):
agent none
```

🏭 Real-World Production Example

Capital One's Jenkins at Scale: Capital One runs Jenkins with 3 Controllers (split by business unit) and 800+ dynamic agents provisioned via the Kubernetes plugin. The Controller has 0 executors — all builds run on Kubernetes pods. Agent pods are labeled by capability: `java11`, `java17`, `python3`, `node16`, `terraform`, `docker`. Each team's Jenkinsfile specifies label expressions. The Controller serves only as orchestrator, and Controller JVM memory usage stays predictable at ~4GB even under heavy build load.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ Why should the Controller have 0 executors in production?

> Two reasons: security and stability. Security: pipeline code runs as Groovy in the Jenkins JVM. If the Controller runs builds, a malicious or buggy pipeline script can access Controller's filesystem (JENKINS_HOME), modify configurations, exfiltrate credentials, or crash the Controller process. With 0 executors, the Controller never executes user code — it only orchestrates. Stability: when the Controller JVM is stressed by builds, the build scheduler, queue management, webhook processing, and UI all slow down. This cascades: builds queue, developers get impatient, and you have an availability incident. The Controller should have predictable, low, steady resource usage. All variable load goes to agents.

Q2: What happens when a Jenkins agent goes offline mid-build?

> The Controller detects the agent disconnection (the Remoting channel drops). The behavior depends on the pipeline configuration: by default, the build is marked as FAILURE with "Agent went offline" error. With `options { retry(3) }`, it can automatically restart. With proper durability settings (`PERFORMANCE_OPTIMIZED` vs `SURVIVABLE_NONATOMIC`), the Controller may retain pipeline state. For Kubernetes agents (ephemeral pods), pod eviction is handled by the K8s plugin — it can reschedule the pod and resume. The key is: the Controller itself is unaffected by agent failures. Only the build running on that agent is impacted.

Q3: How do you route a specific job to a specific type of build environment?

> Through label expressions. I apply labels to agents based on their capabilities — OS, installed tools, available resources. For example: a Java 11 build agent gets labels `linux java java11 maven`. A job that needs Java 11 specifies `agent { label 'java11' }`. A Docker build job specifies `agent { label 'docker' }`. For complex environments, I compose labels: `agent { label 'linux && docker && java11' }`. This way, jobs run on compatible agents without hardcoding agent hostnames — any agent matching the label expression can run the job, enabling load balancing and agent pool scaling.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **"Built-in Node" in modern Jenkins.** Jenkins 2.x renamed "master" to "controller" and the default built-in agent is called "Built-In Node." If you've just installed Jenkins, it has 2 executors on the built-in node by default. **First production action: set built-in node executors to 0.** Many teams forget this and run builds on the controller.
- **Label expression typos cause builds to queue forever.** If you specify `agent { label 'lniux' }` (typo: `lniux` instead of `linux`), Jenkins won't find a matching agent. The build stays in the queue indefinitely with no error — it just waits. Always validate label expressions and monitor for "stuck in queue" builds.
- **Agent connection storms.** When you restart a Jenkins Controller, all agents try to reconnect simultaneously. With 100+ agents, this creates a reconnection storm that can OOM the Controller. Stagger agent reconnections or use connection retries with exponential backoff.
- **Executor count ≠ CPU count.** Some teams set executor count to 32 on a 4-core agent, reasoning "it's IO bound." This works for IO-bound builds but causes CPU contention for compile-heavy builds. Benchmark your typical build and set executors to CPU_cores * 1.5 for IO-heavy, CPU_cores for compile-heavy.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Agent Types (2.4)	Detail of each agent type — SSH, Docker, Kubernetes, JNLP
Jenkins at Scale (2.8)	Controller/agent architecture determines scaling strategy
Jenkins HA (2.7)	Controller is SPOF — HA mitigations start with understanding this architecture
Pipeline Syntax (3.x)	`agent` directive in Declarative Pipeline targets agents by label
Kubernetes Plugin (5.5)	Implements dynamic agent provisioning on Kubernetes
Jenkins Home Directory (2.6)	JENKINS_HOME lives on Controller — understanding its structure is critical

---

Topic 2.2 — Jenkins Installation, System Configuration, and Global Tools

🟢 Beginner | Operational Foundation

📌 What It Is — In Simple Terms

Jenkins installation and system configuration covers: how Jenkins is installed and run, how the system-level settings are configured (JVM, security, URLs, email), how globally available tools (JDK, Maven, Git, NodeJS) are configured so agents can use them, and how these configurations are managed as code rather than through the GUI.

This topic matters in interviews because it tests whether you've actually administered Jenkins versus just used it as a developer. Knowing *where* Jenkins stores things and *how* to configure it from scratch reveals operational depth.

🔍 Why It Matters — The Problem It Solves

Configuration done wrong at the system level cascades into every pipeline:

- JVM configured with insufficient heap → OutOfMemoryErrors during heavy builds
- No tool configuration → pipelines fail with "mvn: command not found"
- No CSRF protection → security vulnerability
- No correct URL configured → email notifications have broken links, webhooks fail
- Manual GUI configuration → can't reproduce after disaster, not version-controlled

---

⚙️ How It Works — Installation Methods

Method 1: WAR File (Bare Metal / VM)

```
# Download Jenkins WAR
wget https://get.jenkins.io/war-stable/latest/jenkins.war

# Run with custom JVM settings (CRITICAL for production)
export JAVA_OPTS="-Xms2g -Xmx4g \
  -XX:+UseG1GC \
  -XX:MaxGCPauseMillis=200 \
  -Djava.awt.headless=true \
  -Djenkins.install.runSetupWizard=false"  # skip setup wizard for
automated installs

export JENKINS_HOME=/var/lib/jenkins

java $JAVA_OPTS -jar jenkins.war \
  --httpPort=8080 \
  --prefix=/jenkins \   # if running behind a proxy at /jenkins
  --logfile=/var/log/jenkins/jenkins.log

# Production: run as systemd service
# /etc/systemd/system/jenkins.service:
```

```
[Unit]
Description=Jenkins Automation Server
After=network.target

[Service]
Type=simple
User=jenkins
Group=jenkins
Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64"
Environment="JENKINS_HOME=/var/lib/jenkins"
Environment="JAVA_OPTS=-Xms2g -Xmx4g -XX:+UseG1GC -XX:MaxGCPauseMillis=200
\
  -Djenkins.install.runSetupWizard=false \
  -Dhudson.model.DirectoryBrowserSupport.CSP=\"sandbox; default-src
'none'; img-src 'self'; style-src 'self';\""
ExecStart=/usr/bin/java $JAVA_OPTS -jar /usr/share/jenkins/jenkins.war \
  --httpPort=8080 \
  --webroot=/var/cache/jenkins/war
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Method 2: Docker (Most Common for Modern Deployments)

```
# Basic Docker run:
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \  # Mount Docker socket
for Docker builds
  --restart=unless-stopped \
  jenkins/jenkins:lts-jdk17

# Production Docker run with JVM tuning:
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -e JAVA_OPTS="-Xms2g -Xmx4g -XX:+UseG1GC -
Djenkins.install.runSetupWizard=false" \
  -e JENKINS_OPTS="--argumentsRealm.passwd.admin=adminpass --
argumentsRealm.roles.admin=admin" \
  -v /data/jenkins_home:/var/jenkins_home \
  --cpus="2" \
  --memory="6g" \
```

```
  --restart=unless-stopped \
  jenkins/jenkins:lts-jdk17
```

Method 3: Kubernetes (Production Standard)

```yaml
# jenkins-deployment.yaml — Jenkins on Kubernetes
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins
  namespace: jenkins
spec:
  replicas: 1  # Always 1 — Jenkins has no native HA
  selector:
    matchLabels:
      app: jenkins
  template:
    metadata:
      labels:
        app: jenkins
    spec:
      serviceAccountName: jenkins  # SA with permissions to create pods
(for K8s agents)
      securityContext:
        runAsUser: 1000    # jenkins user
        fsGroup: 1000
      containers:
      - name: jenkins
        image: jenkins/jenkins:lts-jdk17
        ports:
        - containerPort: 8080    # Web UI
        - containerPort: 50000   # Agent JNLP port
        env:
        - name: JAVA_OPTS
          value: >-
            -Xms2g -Xmx4g
            -XX:+UseG1GC
            -XX:MaxGCPauseMillis=200
            -Djenkins.install.runSetupWizard=false
            -Dcasc.jenkins.config=/var/jenkins_home/casc_configs
        - name: CASC_JENKINS_CONFIG
          value: /var/jenkins_home/casc_configs
        resources:
          requests:
            cpu: "1"
            memory: "3Gi"
          limits:
            cpu: "2"
            memory: "6Gi"
        volumeMounts:
        - name: jenkins-home
```

```
            mountPath: /var/jenkins_home
          - name: casc-config
            mountPath: /var/jenkins_home/casc_configs
          readinessProbe:
            httpGet:
              path: /login
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /login
              port: 8080
            initialDelaySeconds: 90
            periodSeconds: 30
            failureThreshold: 5
        volumes:
        - name: jenkins-home
          persistentVolumeClaim:
            claimName: jenkins-pvc
        - name: casc-config
          configMap:
            name: jenkins-casc-config
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins-pvc
  namespace: jenkins
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi    # Adjust based on build history retention
  storageClassName: ssd  # Use fast storage — JENKINS_HOME is IO-intensive
```

Jenkins Configuration as Code (JCasC) — The Right Way

```
# jenkins.yaml — complete Jenkins system configuration as code
# Applied via CASC_JENKINS_CONFIG env var or
/var/jenkins_home/casc_configs/

jenkins:
  systemMessage: "Jenkins configured via JCasC. Manual config changes will
be overwritten."

  # CRITICAL: Controller executors must be 0 in production
  numExecutors: 0
  mode: EXCLUSIVE
```

```yaml
  # URL configuration (used in notifications, build links)
  rootUrl: "https://jenkins.example.com/"

  # Security configuration
  securityRealm:
    ldap:
      configurations:
      - server: "ldaps://ldap.example.com:636"
        rootDN: "dc=example,dc=com"
        userSearchBase: "ou=users"
        userSearch: "(uid={0})"
        groupSearchBase: "ou=groups"
        managerDN: "cn=jenkins,ou=service-accounts,dc=example,dc=com"
        managerPasswordSecret: "${LDAP_MANAGER_PASSWORD}"  # injected from
env

  authorizationStrategy:
    roleBased:
      roles:
        global:
        - name: "admin"
          description: "Jenkins administrators"
          permissions:
          - "Overall/Administer"
          entries:
          - user: "jenkins-admins"
        - name: "developer"
          description: "Regular developers"
          permissions:
          - "Overall/Read"
          - "Job/Build"
          - "Job/Read"
          - "Job/Workspace"
          entries:
          - group: "engineering"

  # Agent TCP port (for JNLP agents)
  slaveAgentPort: 50000

  # Nodes — static agents defined here
  nodes:
  - permanent:
      name: "build-agent-linux-1"
      labelString: "linux docker java maven"
      numExecutors: 4
      remoteFS: "/home/jenkins/workspace"
      launcher:
        ssh:
          host: "build-agent-1.internal.example.com"
          port: 22
          credentialsId: "ssh-agent-key"
          launchTimeoutSeconds: 60
          maxNumRetries: 3
          retryWaitTime: 30
```

```yaml
# Global tool configuration — tools available on all agents
tool:
  jdk:
  - name: "JDK-17"
    home: "/usr/lib/jvm/java-17-openjdk-amd64"
  - name: "JDK-11"
    home: "/usr/lib/jvm/java-11-openjdk-amd64"

  maven:
  - name: "Maven-3.9"
    home: "/usr/share/maven"

  git:
  - name: "Default"
    home: "/usr/bin/git"

  nodejs:    # requires NodeJS plugin
  - name: "NodeJS-18"
    version: "18.19.0"
    npmPackages: "yarn@1.22.19"   # globally installed npm packages

# Credentials — managed as code (values injected from env/secrets)
credentials:
  system:
    domainCredentials:
    - credentials:
      - usernamePassword:
          id: "github-credentials"
          description: "GitHub service account"
          username: "jenkins-ci-bot"
          password: "${GITHUB_TOKEN}"
      - string:
          id: "sonarqube-token"
          description: "SonarQube authentication token"
          secret: "${SONARQUBE_TOKEN}"
      - aws:
          id: "aws-production"
          description: "AWS production credentials"
          accessKey: "${AWS_ACCESS_KEY_ID}"
          secretKey: "${AWS_SECRET_ACCESS_KEY}"
      - sshUserPrivateKey:
          id: "ssh-agent-key"
          description: "SSH key for agent connections"
          username: "jenkins"
          privateKeySource:
            directEntry:
              privateKey: "${SSH_AGENT_PRIVATE_KEY}"

# Notification configuration
unclassified:
  slackNotifier:
    teamDomain: "mycompany"
    tokenCredentialId: "slack-token"
```

```
      room: "#jenkins-builds"

    mailer:
      smtpHost: "smtp.example.com"
      smtpPort: "587"
      useSsl: false
      useTls: true
      defaultSuffix: "@example.com"

    # SonarQube server configuration
    sonarGlobalConfiguration:
      buildWrapperEnabled: true
      installations:
      - name: "SonarQube"
        serverUrl: "https://sonarqube.example.com"
        credentialsId: "sonarqube-token"
```

JVM Tuning — Critical for Production

```
# Recommended JVM settings for Jenkins Controller (4-8GB RAM machine):
JAVA_OPTS="\
  -Xms2g \                    # Initial heap — set equal to Xmx to avoid
GC pressure from heap growth
  -Xmx4g \                    # Max heap — leave headroom for OS and non-
heap
  -XX:+UseG1GC \              # G1 GC — best for large heaps with latency
targets
  -XX:MaxGCPauseMillis=200 \  # Target max GC pause — keeps UI responsive
  -XX:+ExplicitGCInvokesConcurrent \  # System.gc() uses concurrent G1
instead of full GC
  -XX:+UnlockDiagnosticVMOptions \
  -XX:G1NewSizePercent=20 \   # Young gen sizing for G1
  -XX:G1MaxNewSizePercent=40 \
  -XX:G1HeapRegionSize=8m \   # Larger regions for larger heap
  -Djava.awt.headless=true \  # No display needed for server
  -Dfile.encoding=UTF-8 \
  -Dhudson.model.DownloadService.noSignatureCheck=false \  # verify plugin
signatures
  -Djenkins.install.runSetupWizard=false"  # skip wizard in automated
deployments

# For very large Jenkins (>8GB heap), consider ZGC (Java 15+):
# -XX:+UseZGC -XX:SoftMaxHeapSize=6g
```

Nginx Reverse Proxy Configuration (Production Standard)

```
# /etc/nginx/sites-enabled/jenkins
upstream jenkins {
```

```
    server 127.0.0.1:8080 fail_timeout=0;
}

server {
    listen 443 ssl http2;
    server_name jenkins.example.com;

    ssl_certificate     /etc/ssl/certs/jenkins.example.com.crt;
    ssl_certificate_key /etc/ssl/private/jenkins.example.com.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
    ssl_ciphers         HIGH:!aNULL:!MD5;

    # Required for Jenkins reverse proxy
    location / {
        proxy_pass         http://jenkins;
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto https;   # CRITICAL for
Jenkins URL detection
        proxy_set_header   X-Forwarded-Port 443;
        proxy_read_timeout 90s;

        # For WebSocket support (required for Blue Ocean, agents)
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection "upgrade";
    }
}

server {
    listen 80;
    server_name jenkins.example.com;
    return 301 https://$host$request_uri;
}
```

🔑 Key Concepts

Concept	Meaning
JENKINS_HOME	Root directory storing all config, jobs, builds, plugins
JCasC	Jenkins Configuration as Code plugin — configure Jenkins via YAML
Global Tools	Tools (JDK, Maven, Git) configured at system level, available to all pipelines
Security Realm	Authentication backend — LDAP, GitHub OAuth, Jenkins DB, SAML
Authorization Strategy	Access control — Role-Based (RBAC), Matrix, Project-based
CSRF Protection	Cross-Site Request Forgery protection — always enabled in production
Agent TCP Port 50000	Port agents use for JNLP connections to Controller

Concept	Meaning
JVM Heap	Memory allocated to Jenkins JVM — must be sized for build load
G1GC	Garbage First GC — recommended for Jenkins at medium/large heaps
Crumb	Jenkins CSRF token — required for API calls from external systems

---

💬 Short Crisp Interview Answer

> *"Jenkins can be installed as a WAR file, Docker container, Helm chart, or via package managers like apt/yum. The critical system configurations for production are: JVM heap sizing with G1GC to prevent OutOfMemoryErrors and long GC pauses; setting Controller executors to 0 so builds never run on the Controller; configuring the correct Jenkins URL so webhooks and notifications work; setting up an authentication backend like LDAP or SSO; and configuring global tools like JDK and Maven versions so pipelines can reference them by name. In production, all of this is managed via Jenkins Configuration as Code — JCasC plugin — so the entire Jenkins configuration is a YAML file in version control that can recreate the system from scratch."*

🔬 Deep Dive Answer

Plugin Management at Scale:

```
# Install plugins non-interactively (for automated Jenkins setup):
# Method 1: jenkins-plugin-cli (recommended):
jenkins-plugin-cli --plugins \
  pipeline-model-definition:latest \
  git:latest \
  kubernetes:latest \
  configuration-as-code:latest \
  workflow-aggregator:latest \
  blueocean:latest \
  sonar:latest \
  -d /usr/share/jenkins/ref/plugins/

# Method 2: Via Groovy init script (runs on startup):
# /var/jenkins_home/init.groovy.d/install-plugins.groovy
import jenkins.model.*
import hudson.PluginManager

def plugins = [
    'git', 'workflow-aggregator', 'kubernetes',
    'configuration-as-code', 'credentials-binding'
]

def pm = Jenkins.instance.pluginManager
def uc = Jenkins.instance.updateCenter
uc.updateAllSites()

plugins.each { plugin ->
```

```
    if (!pm.getPlugin(plugin)) {
        def p = uc.getPlugin(plugin)
        if (p) {
            p.deploy(true)
        }
    }
}
Jenkins.instance.restart()
```

Proxy Configuration for Plugin Downloads:

```
# Jenkins downloads plugins from update centers — may need proxy in
restricted networks
# Configure in Manage Jenkins → Plugin Manager → Advanced → HTTP Proxy

# Via JVM system properties:
JAVA_OPTS="... \
  -Dhttp.proxyHost=proxy.example.com \
  -Dhttp.proxyPort=3128 \
  -Dhttps.proxyHost=proxy.example.com \
  -Dhttps.proxyPort=3128 \
  -Dhttp.nonProxyHosts=localhost|*.internal.example.com"
```

Security Hardening Checklist:

```
✅ Controller executors = 0
✅ Enable CSRF protection (enabled by default in Jenkins 2.x)
✅ Disable CLI over remoting (Manage Jenkins → Security → Agent → Disable
CLI)
✅ Enable "Agent → Controller Security" (sandbox for agent-to-controller
file operations)
✅ Configure Content Security Policy for HTML reports
✅ Use HTTPS (TLS 1.2+ only)
✅ Disable signup (Manage Jenkins → Security → Allow users to sign up →
unchecked)
✅ Use Role-Based Authorization (RBAC plugin) not "anyone can do anything"
✅ Restrict job-to-job credentials access (credential scope: System vs
Global)
✅ Enable audit logging (Audit Trail plugin)
✅ Restrict script approval for Groovy pipelines
```

🏭 Real-World Production Example

ING Bank's Jenkins Configuration: ING Bank manages 50+ Jenkins instances across business units using JCasC + Helm. Every Jenkins instance is defined by a `values.yaml` file in a Helm chart that includes the JCasC configuration. Spinning up a new Jenkins for a team takes 10 minutes via `helm install`. When security requirements change (new LDAP attribute for RBAC), they update one Helm values template and

roll it out to all 50 instances via GitOps. No manual GUI configuration exists — a Jenkins that requires GUI configuration is treated as a drift incident.

❓ Common Interview Questions & Strong Answers

Q1: What happens if Jenkins runs out of heap memory?

> The Jenkins JVM throws `OutOfMemoryError`, the process crashes, and Jenkins becomes unavailable until restarted. All in-flight builds are lost. The causes are: insufficient `-Xmx` setting for the build load, memory leaks in plugins (common in poorly maintained plugins), build log streams not being closed, or workspace data being held in memory. Prevention: set `-Xms` equal to `-Xmx` to eliminate heap resize GC, use G1GC with a target max pause time, monitor heap usage via JMX or Prometheus JVM metrics, and configure automatic Controller restart on OOM (`-XX:+ExitOnOutOfMemoryError -XX:OnOutOfMemoryError="systemctl restart jenkins"`).

Q2: How do you configure Jenkins without touching the GUI?

> JCasC — Jenkins Configuration as Code plugin. All system configuration — security settings, credentials, agent definitions, tool configurations, plugin settings — is expressed as YAML. The YAML is stored in version control. Jenkins reads it from a directory specified by `CASC_JENKINS_CONFIG` environment variable at startup. Any changes go through git commit + Jenkins restart (or live reload via `/configuration-as-code/reload` endpoint). This means Jenkins configuration is versionable, reviewable, and reproducible. A completely fresh Jenkins can be configured to production state in minutes by pointing it at the JCasC YAML.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Jenkins URL misconfiguration.** If Jenkins Root URL is set to `http://localhost:8080` (default), webhook callbacks from GitHub fail (Jenkins can't be reached at localhost from GitHub), email notifications have broken links, and Blue Ocean can't construct URLs. Always set Root URL to the public/resolvable FQDN. With JCasC: `jenkins.rootUrl: "https://jenkins.example.com/"` (trailing slash required).
- **Plugin update instability.** Jenkins plugins update frequently and sometimes introduce breaking changes. In production, pin plugin versions explicitly rather than using `:latest` in automated installs. Test plugin updates in a staging Jenkins before applying to production.
- **`/var/jenkins_home` on slow storage.** Jenkins reads/writes JENKINS_HOME constantly — job configs, build logs, plugin files. If JENKINS_HOME is on a network file system (NFS with high latency), Jenkins becomes sluggish. Use local SSD storage or fast block storage (EBS gp3, GCP Persistent SSD). IOPS matters more than throughput.
- **JCasC credential security.** JCasC YAML stored in Git must NOT contain plaintext secrets. Use `${ENV_VAR}` placeholders — JCasC resolves them from environment variables at startup. Inject actual secret values via Kubernetes Secrets, Vault, or parameter store.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Home Directory (2.6)	Installation creates and populates JENKINS_HOME
Jenkins HA (2.7)	Backup strategy depends on understanding JENKINS_HOME
JCasC Plugin (5.7)	The configuration-as-code plugin makes system config reproducible
Jenkins at Scale (2.8)	JVM tuning directly impacts Controller scalability
Security Plugins (5.4)	Auth strategy and RBAC configured at system level
Kubernetes Integration (6.3)	K8s deployment method for Jenkins controller

---

Topic 2.3 — Jenkins Controller Internals: Job Queue, Build Queue, Thread Model

🟡 Intermediate | Under The Hood

📌 What It Is — In Simple Terms

This topic covers what happens **inside the Jenkins Controller** after a build is triggered: how builds are queued, how the scheduler assigns builds to agents, what threads are running, and what can cause the Controller to become slow or unresponsive. Understanding this separates Jenkins users from Jenkins operators.

🔍 Why This Matters — The Problem It Solves

When developers complain "Jenkins is slow" or "my build has been queued for 30 minutes," the cause is almost always inside the Controller internals — queue management, thread starvation, GC pressure, or blocked executors. You can't diagnose or fix these without understanding the internals.

⚙️ How It Works — The Full Internal Flow

The Jenkins Execution Model

```
┌──────────────────────────────────────────────────────────────────┐
│                    JENKINS CONTROLLER JVM                        │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                    TRIGGER SOURCES                       │   │
│  │  SCM Polling │ GitHub Webhook │ Cron │ Upstream │ Manual  │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                  │                               │
│                                  ↓                               │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │                     BUILD QUEUE                          │   │
```

```
│   │                                                                │    │
│   │  Item 1: myapp-pipeline #46  [waiting: no agent available]  │    │
│   │  Item 2: auth-service  #23   [waiting: quiet period]        │    │
│   │  Item 3: test-suite    #11   [waiting: blocked by #10]      │    │
│   │                                                              │    │
│   │  Queue states:                                               │    │
│   │    WAITING  → quiet period not expired                       │    │
│   │    BLOCKED  → waiting for other build to finish              │    │
│   │    BUILDABLE → ready to run, waiting for executor            │    │
│   └──────────────────────────────────────────────────────────────┘    │
│                                  │                                     │
│                                  ↓                                     │
│   ┌──────────────────────────────────────────────────────────────┐    │
│   │                     SCHEDULER THREAD                         │    │
│   │  (runs every ~200ms)                                         │    │
│   │                                                              │    │
│   │  For each BUILDABLE item:                                    │    │
│   │    1. Find agents matching label expression                  │    │
│   │    2. Find agent with free executor                          │    │
│   │    3. Select least-loaded matching agent                     │    │
│   │    4. Dispatch build to agent                                │    │
│   └──────────────────────────────────────────────────────────────┘    │
│                                  │                                     │
│                                  ↓                                     │
│   ┌──────────────────────────────────────────────────────────────┐    │
│   │                   THREAD POOLS                               │    │
│   │                                                              │    │
│   │  Computer.threadPoolForRemoting  → agent communication       │    │
│   │  Executor threads               → build coordination         │    │
│   │  Timer (Trigger) threads        → SCM polling, cron          │    │
│   │  Jetty HTTP threads             → Web UI, API, webhooks      │    │
│   │  AsyncPeriodicWork threads      → plugin background tasks    │    │
│   └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

The Build Queue States in Detail

```
State: WAITING
  → Quiet period not yet expired (default 5 seconds after trigger)
  → Purpose: deduplicate(remove duplicates) rapid successive commits
  → If same job triggered 3 times in 2 seconds → one build runs
  → Configurable: quietPeriod(0) to disable, or higher values to batch

State: BLOCKED
  → A dependency build hasn't completed
  → Build has concurrency limits (throttle plugin)
  → Build requires a "Lockable Resource" that's in use

State: BUILDABLE
  → All conditions met, ready to run
  → Waiting only for a free executor matching label requirements
  → THIS is where "stuck in queue" issues occur if no agent available
```

```
State: PENDING (Kubernetes/cloud agents only)
  → New agent is being provisioned (pod starting, VM spinning up)
  → Jenkins is waiting for the new agent to come online
```

The Quiet Period — Deduplication Mechanism

```
// Quiet Period: prevents duplicate builds from rapid triggers
// Default: 5 seconds after trigger → if same job triggered again
// within quiet period, triggers are merged into one build

// Configure in pipeline:
pipeline {
    options {
        quietPeriod(0)  // Disable quiet period — build immediately
                        // Use for urgent builds (hotfixes)
    }
}

// Or: increase quiet period for batch jobs
options {
    quietPeriod(30)  // Wait 30s — all commits in a push batch into one
build
}
```

Throttle Concurrent Builds — Build Queue Blocking

```
// Prevent too many concurrent builds of the same job
pipeline {
    options {
        // Max 2 concurrent builds of this pipeline
        throttleJobProperty(
            categories: [],
            limitOneJobWithMatchingParams: false,
            maxConcurrentPerNode: 0,
            maxConcurrentTotal: 2,      // ← global limit
            throttleEnabled: true,
            throttleOption: 'project'
        )
    }
}
```

Jenkins Thread Model — What Threads Are Running

```
# View Jenkins thread dump (critical for diagnosing hangs/slowness):
# Method 1: Thread dump endpoint (admin only):
curl -s https://jenkins.example.com/threadDump -u admin:${ADMIN_TOKEN}
```

```
# Method 2: jstack on Jenkins JVM:
jstack $(pgrep -f jenkins.war) > /tmp/jenkins-threads.txt

# Key threads to look for in a thread dump:

# HEALTHY Controller:
"Executor #5 for build-agent-1"      → RUNNABLE (running a build)
"Jenkins scheduler"                   → TIMED_WAITING (normal — sleeping
between checks)
"Jetty-QTP-handler-0"                → RUNNABLE (serving HTTP requests)
"SCM polling trigger"                 → TIMED_WAITING (waiting for next
poll)

# UNHEALTHY Controller (signs of trouble):
"Jenkins scheduler"  → BLOCKED (waiting on a monitor) ← scheduler hung
"Jetty-QTP-handler-[0-50]"  → all BLOCKED ← HTTP handler exhaustion (UI
slow)
"Finalizer"                 → RUNNABLE with huge finalizer queue ← GC
issue
Many threads in "hudson.remoting" → agent connection storm
```

Controller HTTP Thread Pool

```
Jenkins uses Jetty as its embedded web server.
Jetty has a thread pool for handling HTTP requests.

Default: ~200 HTTP threads

Problem: Each Groovy step in a running pipeline ties up a controller
thread
(waiting for the step result from the agent)

With 100 concurrent builds, 100 threads blocked waiting for agent
responses
→ Only 100 threads left for UI, webhooks, API calls
→ Developers see: slow UI, webhooks timing out, API calls failing

Solution 1: Increase Jetty thread pool (temporary):
  JAVA_OPTS="... -Dhudson.TcpSlaveAgentListener.connectionTimeout=15"

Solution 2: Enable pipeline durability optimization:
  options { durabilityHint('PERFORMANCE_OPTIMIZED') }
  → Reduces controller thread usage per pipeline step

Solution 3: Reduce concurrent builds per controller
  → Add more controllers (federated Jenkins)
```

SCM Polling vs Webhooks — Internal Behavior

```
SCM Polling (legacy, avoid in production):
  → Timer thread wakes every N minutes (e.g., "H/5 * * * *")
  → For each job with polling configured:
      git ls-remote <repo-url>  ← HTTP request from Controller to Git
server
  → With 200 jobs polling every 5 min:
      200 requests × 12 times/hour = 2,400 requests/hour FROM Controller
  → Scales poorly: Controller network + CPU thrashing
  → Latency: up to 5 minutes before build starts after push

Webhooks (correct approach):
  → Git server pushes notification to Jenkins Controller URL
  → Jenkins receives POST /github-webhook/ or /generic-webhook-
trigger/invoke
  → One Jetty thread handles the webhook → triggers build → thread
released
  → No polling load on Controller
  → Latency: build starts within seconds of push
  → Jenkins URL must be reachable from Git server (firewall consideration)
```

🔑 Key Concepts

Concept	Meaning
Build Queue	In-memory queue of builds waiting to run on the Controller
Quiet Period	Wait time after trigger before build runs — allows deduplication
Scheduler Thread	Controller thread that dispatches BUILDABLE items to free executors
BLOCKED state	Build waiting for a dependency, concurrency limit, or lockable resource
BUILDABLE state	Build ready to run but no free executor available — "stuck in queue"
Throttle Plugin	Limits concurrent builds globally or per-node
SCM Polling	Controller-initiated polling of Git — scales poorly, avoid in production
Webhook	Git server pushes to Jenkins — preferred over polling
Thread Dump	Snapshot of all JVM threads — essential for diagnosing Controller hangs
Jetty Thread Pool	HTTP threads serving UI, API, webhooks — can become exhausted under load

---

💬 Short Crisp Interview Answer

> *"When a build is triggered, Jenkins adds it to the Build Queue. The queue has three states: WAITING (quiet period), BLOCKED (dependency or concurrency limit), and BUILDABLE (ready, waiting for a free executor). The Controller's scheduler thread runs roughly every 200 milliseconds, checking for BUILDABLE items and matching them to agents with free executors based on label expressions. The Controller runs multiple thread pools: Jetty threads for HTTP, executor threads for build*

> *coordination, timer threads for polling and cron triggers, and remoting threads for agent communication. Performance problems occur when the Jetty thread pool is exhausted by concurrent builds blocking Controller threads, when SCM polling creates too many outbound connections, or when GC pressure from large heaps causes pauses that block the scheduler. The correct diagnosis tool is a thread dump — `/threadDump` endpoint or `jstack`."*

🔬 Deep Dive Answer

The CPS (Continuation Passing Style) Pipeline Engine:

This is a deep internals topic that almost no candidate knows — it impresses interviewers.

```
Jenkins Declarative and Scripted Pipelines are not executed as regular
Groovy.
They are compiled to CPS (Continuation Passing Style) bytecode.

Why CPS?
  Normal Java/Groovy code: execution state lives on the call stack
  If the JVM dies, all state is lost

  CPS transforms code so execution state is SERIALIZABLE:
  Each "step" becomes a Continuation — a snapshot of "what to do next"
  Continuations are saved to
JENKINS_HOME/jobs/<job>/builds/<n>/program.dat

  If Jenkins restarts mid-build:
  → Load program.dat → resume from last checkpoint
  → This is pipeline durability

The cost:
  Every step serializes state to disk → IO overhead
  Complex Groovy (closures, recursive calls) can fail CPS transformation
  "@NonCPS" annotation bypasses CPS (for utility methods) — loses
durability

  // @NonCPS: runs as normal Groovy (fast, not serialized, no durability)
  @NonCPS
  def parseComplexData(String raw) {
      // Complex string manipulation, regex, list operations
      // Cannot be paused/resumed — use @NonCPS for utility methods
      return raw.split('\n').collect { it.trim() }.findAll { it }
  }
```

Pipeline Durability Levels:

```
// Higher durability = more disk IO = slower builds
// Lower durability = less disk IO = faster builds, but no recovery on
controller crash
```

```
pipeline {
    options {
        // Options from most durable to least:
        durabilityHint('MAX_SURVIVABILITY')          // Write to disk
after EVERY step
                                                     // Slowest, most
resilient

        durabilityHint('SURVIVABLE_NONATOMIC')       // Default — writes
frequently

        durabilityHint('PERFORMANCE_OPTIMIZED')      // Write at stage
boundaries only
                                                     // Fastest — 2-3x
throughput improvement
                                                     // Use for most
production pipelines
                                                     // Risk: on
Controller crash, lose progress
                                                     // within current
stage
    }
}
```

Build Queue Stuck Analysis:

```
# Diagnose why a build is stuck in queue:

# Jenkins API — get queue details:
curl -s https://jenkins.example.com/queue/api/json?pretty=true \
  -u admin:${TOKEN} | python3 -m json.tool

# Look for:
# "why": "Waiting for next available executor on nodes with label
'docker'"
#   → All docker-labeled agents are busy OR no docker agents connected

# "why": "Build #45 is already in progress"
#   → Job has disableConcurrentBuilds() — previous build must finish

# "why": "Waiting for Throttle to allow running"
#   → Throttle plugin limit reached

# "why": "All nodes of label 'linux' are offline"
#   → All matching agents are disconnected — infrastructure problem
```

🏭 Real-World Production Example

Spotify's Build Queue Optimization: Spotify had a recurring issue where their Jenkins Controller became unresponsive during peak build times (post-standup commits). Root cause analysis via thread dump: 180 of 200 Jetty threads were blocked waiting for agent responses from pipeline steps. Resolution:

1. Switched all pipelines to `PERFORMANCE_OPTIMIZED` durability — reduced per-step Controller thread blocking
2. Increased Jetty thread pool via `-Djava.util.concurrent.ForkJoinPool.common.parallelism=256`
3. Migrated all SCM polling to GitHub webhooks — eliminated 1,200 polling requests/hour
4. Set quiet period to 10 seconds — deduplicated batch commit triggers Result: Controller CPU dropped 60%, UI response time improved 10x during peak hours.

❓ Common Interview Questions & Strong Answers

Q1: A build has been in the queue for 30 minutes. How do you diagnose and fix it?

> First, check the queue via UI or API — the "why" field in the queue item tells you the exact reason: no available executor, concurrency limit, blocked by previous build, or no matching agent. If "no matching agent": check if agents with the required label are connected (`/manage/nodes`), if not connected investigate agent health. If connected but busy: check executor utilization, add more agents or increase executor count. If throttle plugin limit: evaluate if limit needs adjustment. If blocked by previous build: check if that build is hung and might need to be aborted. For systematic issues: check if quiet period is set too high, or if builds are not completing and filling agent executors.

Q2: What is the CPS transformation and why does it matter for pipeline development?

> CPS — Continuation Passing Style — is how Jenkins transforms pipeline Groovy code to make execution state serializable to disk, enabling pipeline durability. Every pipeline step is converted to a continuation that can be saved and resumed after a Controller restart. The practical implication for developers: not all Groovy constructs work in CPS context (recursion, complex closures, Java serialization of local objects). When a pipeline step fails with "unable to serialize" errors, you've hit a CPS limitation. The fix is `@NonCPS` annotation on utility methods — but those methods lose durability. Understanding CPS explains why pipelines use explicit `readFile`, `writeFile` steps for file operations instead of Groovy's native file APIs.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Quiet period deduplication has a gotcha.** If you push 5 commits rapidly, quiet period collapses them into one build. But that one build checks out the HEAD — it doesn't run 5 builds. If any of those 5 commits introduced a bug that was fixed in commit 5, the single build passes and the intermediate bug is invisible. This is usually fine (you care about HEAD), but breaks changelog-based testing approaches.
- **Build queue memory is not bounded.** If webhooks fire faster than builds complete, the queue grows indefinitely. Each queue item holds references to the triggering payload, parameters, etc. A webhook storm (GitHub misconfiguration sending duplicate events) can OOM the Controller through queue growth. Monitor queue depth as a metric and alert above a threshold.

- **Throttle plugin scope confusion.** Throttle has "per-project" and "per-category" modes. Setting throttle on a Multibranch Pipeline project throttles the parent, not individual branch builds. You must configure throttle at the individual branch pipeline level for per-branch limits.
- **SCM polling "H syntax" unpredictability.** `H/5 * * * *` uses a hash of the job name to spread polls within the 5-minute window — it does NOT mean "every 5 minutes on the 0:00, 0:05, 0:10" marks. Two jobs both configured `H/5 * * * *` will poll at different offsets. This is a feature (prevents thundering herd), but confuses engineers expecting aligned polling.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Architecture (2.1)	Queue and scheduler are parts of Controller internals
Jenkins at Scale (2.8)	Queue depth and thread pool exhaustion are key scaling bottlenecks
Pipeline Durability (4.7)	CPS engine + durability hints directly relate to this topic
Agent Types (2.4)	Queue BUILDABLE state resolves when agents have free executors
Pipeline Triggers (3.9)	How builds enter the queue — polling, webhooks, cron
Parallel Stages (3.7)	Parallel stages spawn multiple queue items simultaneously

---

Topic 2.4 — Agent Types: Static, Dynamic, Docker, Kubernetes, SSH, JNLP

🟡 Intermediate | Execution Infrastructure

📌 What It Is — In Simple Terms

A Jenkins **Agent** is any machine, container, or compute resource that the Controller delegates build execution to. There are multiple agent types, each with different connection methods, lifecycle management, and use cases. Choosing the right agent type determines your build infrastructure's scalability, cost, isolation, and maintainability.

🔍 Why Multiple Agent Types Exist

Agent Type	Problem It Solves
Static/Permanent	Persistent, always-available workers for predictable load
SSH	Simple connection to Linux VMs/bare-metal without agent process pre-installed
JNLP/Inbound	Agents behind NAT/firewall where Controller can't initiate SSH
Docker	Per-build clean environment using Docker containers as agents

Agent Type	Problem It Solves
Kubernetes	Elastic, on-demand pod agents that scale to zero between builds
Cloud (EC2/GCP)	Provision and terminate VMs dynamically — pay per build

---

⚙️ Agent Types In Detail

Type 1: Static / Permanent Agents

```
Definition: A fixed machine pre-configured in Jenkins that stays connected
Lifecycle: Persistent — connected 24/7, waiting for builds
Connection: SSH (controller initiates) or JNLP (agent initiates)
Use cases: Dedicated build machines, specialized hardware, licensed tools
```

```yaml
# JCasC static agent definition:
jenkins:
  nodes:
  - permanent:
      name: "build-linux-1"
      labelString: "linux docker java maven"
      numExecutors: 4
      remoteFS: "/home/jenkins/workspace"
      retentionStrategy: "always"   # Keep connected always
      launcher:
        ssh:
          host: "10.0.1.50"
          port: 22
          credentialsId: "ssh-agent-key"
          launchTimeoutSeconds: 60
          maxNumRetries: 5
          retryWaitTime: 15
          javaPath: "/usr/lib/jvm/java-17/bin/java"
          jvmOptions: "-Xms512m -Xmx1g"
```

```
Pros:
  ✅ Always available — no provisioning latency
  ✅ Tools pre-installed — no startup overhead
  ✅ Persistent workspace — can cache dependencies between builds
  ✅ Good for licensed software (tools with per-machine licenses)

Cons:
  ❌ Fixed capacity — can't scale beyond provisioned machines
  ❌ Configuration drift — agents diverge from each other over time
  ❌ Cost — running 24/7 even during off hours
  ❌ Maintenance burden — OS patching, tool version management
```

Type 2: SSH Agents

```
Jenkins Controller SSH's into the agent machine and starts the agent
process:

Controller ──[SSH]──> Agent machine
              → Controller copies agent.jar to remote machine
              → Launches: java -jar agent.jar -jnlpUrl ...
              → Remoting channel established over SSH

Requirements on agent machine:
  - SSH server running and accessible
  - Java installed (same major version as Controller)
  - jenkins user with SSH key configured
  - No Jenkins agent pre-installed — Controller does it
```

```
// In Jenkinsfile — run on SSH agent:
pipeline {
    agent { label 'linux' }  // Targets any agent with 'linux' label
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'  // Runs on the SSH agent
            }
        }
    }
}
```

Type 3: JNLP / Inbound Agents

```
Agent INITIATES connection to Controller (reversed direction):

Agent machine ──[TCP 50000 or WebSocket]──> Jenkins Controller

Why: Agent is behind NAT/firewall where Controller can't SSH in
     Cloud VMs with no public IP can still connect outbound

Agent startup command:
java -jar agent.jar \
  -jnlpUrl https://jenkins.example.com/computer/my-agent/jenkins-
agent.jnlp \
  -secret abc123def456 \  # unique token per agent
  -workDir "/home/jenkins"

# Docker run for JNLP agent:
docker run --init jenkins/inbound-agent \
  -url https://jenkins.example.com \
```

```
  -secret ${AGENT_SECRET} \
  -name "inbound-agent-1" \
  -workDir /home/jenkins/agent
```

Type 4: Docker Agents — Build Inside Containers

```
Jenkins runs each build (or stage) inside a Docker container:
  → Container created at build start
  → Build runs inside container
  → Container destroyed at build end
  → Each build gets a completely fresh environment

Two Docker agent approaches:

A) Docker container as the entire agent:
   agent {
       docker {
           image 'maven:3.9-eclipse-temurin-17'
           args '-v /root/.m2:/root/.m2'  // Mount Maven cache
       }
   }

B) Docker container run INSIDE an agent (Docker-in-Docker or DooD):
   The agent machine has Docker installed
   Pipeline uses 'sh "docker build ..."' to build images
```

```
// Full Docker agent pipeline example:
pipeline {
    agent none  // No global agent — per-stage agents

    stages {
        stage('Build Java') {
            agent {
                docker {
                    image 'maven:3.9-eclipse-temurin-17'
                    label 'docker'  // Run on agents with Docker installed
                    args '--network=host -v /root/.m2:/root/.m2:ro'
                    alwaysPull true  // Always pull latest image
                }
            }
            steps {
                sh 'mvn clean package -DskipTests'
                stash name: 'jar', includes: 'target/*.jar'
            }
        }

        stage('Run Tests in Different JDK') {
            agent {
                docker {
```

```
                    image 'eclipse-temurin:11'  // Different JDK version!
                    label 'docker'
                }
            }
            steps {
                unstash 'jar'
                sh 'java -version && java -jar target/myapp.jar --test'
            }
        }

        stage('Build Docker Image') {
            agent { label 'docker' }  // Use host Docker on this agent
            steps {
                sh 'docker build -t myapp:${GIT_COMMIT[0..6]} .'
                sh 'docker push registry.io/myapp:${GIT_COMMIT[0..6]}'
            }
        }
    }
}
```

Type 5: Kubernetes Agents — The Production Standard ⚠

```
The Kubernetes Plugin provisions a new Pod for each build:

Jenkins Controller
  → Plugin sends: "create pod with these containers"
  → Kubernetes API creates pod in jenkins-agents namespace
  → Pod connects back to Controller via JNLP
  → Build runs in pod
  → Build completes → pod is DELETED

Key features:
  - Pod Template: defines containers, volumes, resources, labels
  - JNLP sidecar: the jnlp container handles Controller communication
  - Custom containers: build tool containers (maven, node, docker)
  - Volume mounts: shared workspace between containers, cache volumes
  - Resource limits: CPU/memory per build pod
  - Node selectors: target specific K8s node pools
  - Tolerations: schedule on tainted nodes (e.g., spot/preemptible
instances)
```

```
// Kubernetes agent in Jenkinsfile — full pod template:
pipeline {
    agent {
        kubernetes {
            label 'build-pod'
            yaml '''
apiVersion: v1
kind: Pod
```

```yaml
  metadata:
    labels:
      jenkins-agent: "true"
  spec:
    # Use spot/preemptible instances for cost savings
    tolerations:
    - key: "spot"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
    nodeSelector:
      cloud.google.com/gke-spot: "true"

    # Never schedule on Controller node
    affinity:
      nodeAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
          nodeSelectorTerms:
          - matchExpressions:
            - key: role
              operator: NotIn
              values: ["jenkins-controller"]

    containers:
    # JNLP container - required, handles Controller communication
    - name: jnlp
      image: jenkins/inbound-agent:latest-jdk17
      resources:
        requests:
          cpu: "100m"
          memory: "256Mi"
        limits:
          cpu: "200m"
          memory: "512Mi"

    # Maven build container
    - name: maven
      image: maven:3.9-eclipse-temurin-17
      command: ["cat"]
      tty: true
      resources:
        requests:
          cpu: "500m"
          memory: "1Gi"
        limits:
          cpu: "2"
          memory: "3Gi"
      volumeMounts:
      - name: maven-cache
        mountPath: /root/.m2/repository

    # Docker-out-of-Docker (DooD) container for building images
    - name: docker
      image: docker:24-dind
```

```
      securityContext:
        privileged: true   # Required for Docker-in-Docker
      volumeMounts:
      - name: docker-socket
        mountPath: /var/run/docker.sock

    # Helm/kubectl for deployments
    - name: helm
      image: alpine/helm:3.13.0
      command: ["cat"]
      tty: true

    volumes:
    # Shared Maven cache via PVC (speeds up builds)
    - name: maven-cache
      persistentVolumeClaim:
        claimName: maven-cache-pvc
    # Docker socket from host (DooD pattern)
    - name: docker-socket
      hostPath:
        path: /var/run/docker.sock
'''
        }
    }

    stages {
        stage('Build') {
            steps {
                container('maven') {          // Run in maven container
                    sh 'mvn clean package'
                }
            }
        }

        stage('Docker Build') {
            steps {
                container('docker') {         // Run in docker container
                    sh 'docker build -t myapp:${GIT_COMMIT[0..6]} .'
                    sh 'docker push registry.io/myapp:${GIT_COMMIT[0..6]}'
                }
            }
        }

        stage('Deploy') {
            steps {
                container('helm') {           // Run in helm container
                    sh 'helm upgrade --install myapp ./chart --set
image.tag=${GIT_COMMIT[0..6]}'
                }
            }
        }
    }
}
```

```
Kubernetes Agent Pros:
  ✅ Scales to zero (no idle cost)
  ✅ Perfectly clean environment per build (ephemeral)
  ✅ Multiple containers per build pod (maven + docker + helm in one
build)
  ✅ Resource limits enforced by Kubernetes
  ✅ Spot/preemptible instances for cost savings
  ✅ No agent maintenance — container images as agents

Kubernetes Agent Cons:
  ❌ Pod startup latency (~30-60s cold start)
  ❌ No workspace caching by default (must use PVCs)
  ❌ Docker-in-Docker (DinD) has security and performance concerns
  ❌ Requires Kubernetes cluster with sufficient resources
  ❌ More complex debugging (pod logs, agent communication issues)
```

🔑 Key Concepts

Concept	Meaning
Permanent Agent	Always-on, pre-configured agent — static capacity
Dynamic Agent	Provisioned on-demand per build, destroyed after — elastic capacity
JNLP (Inbound)	Agent-initiated connection — agent connects OUT to Controller
SSH (Outbound)	Controller-initiated connection — Controller SSH's IN to agent
Pod Template	Kubernetes spec defining the agent pod's containers and resources
DinD	Docker-in-Docker — running Docker daemon inside a container (privileged, security concern)
DooD	Docker-outside-of-Docker — mounting host Docker socket into container (simpler, but security concern)
JNLP Secret	Per-agent authentication token preventing unauthorized connections
Agent Workspace	Working directory on agent where pipeline checkouts and builds run
Cloud Plugin	Jenkins plugin that provisions cloud-based dynamic agents (K8s, EC2, GCP)

💬 Short Crisp Interview Answer

> *"Jenkins has several agent types. Static/Permanent agents are always-on VMs or bare-metal machines connected via SSH (Controller initiates) or JNLP (agent initiates) — good for predictable load and specialized tools. Docker agents run each build in a fresh container on a Docker-enabled host — excellent isolation, clean environments. Kubernetes agents are the production standard — the Kubernetes plugin creates a new Pod per build using a Pod Template YAML, the pod contains build tool containers, runs the job, and is deleted on completion. K8s agents scale to zero between builds and support multiple containers per pod (maven, docker, helm all in one build pod). For production, I prefer Kubernetes agents on spot/preemptible instances for cost efficiency, with PVC-backed Maven/npm caches to offset the cold-start overhead."*

🔬 Deep Dive Answer

Docker-in-Docker (DinD) vs Docker-out-of-Docker (DooD) vs Kaniko:

```
Option 1: Docker-in-Docker (DinD)
  Run a full Docker daemon inside the build container
  Requires: privileged: true  (security risk)

  Pros: Fully isolated Docker environment
  Cons: Privileged containers have cluster-level root access
        Slower (daemon startup per build)
        Storage driver conflicts with outer Docker

Option 2: Docker-out-of-Docker (DooD)
  Mount /var/run/docker.sock from host into container
  Build container shares the host Docker daemon

  Pros: Faster (no daemon startup)
  Cons: /var/run/docker.sock access = root on host node
        Build can see and affect OTHER builds' containers
        Still a security concern but simpler than DinD

Option 3: Kaniko (Recommended for Production K8s)
  Build container images WITHOUT Docker daemon
  Executes Dockerfile steps in userspace
  No privileged required

  container:
  - name: kaniko
    image: gcr.io/kaniko-project/executor:v1.19.0
    command: ["/kaniko/executor"]
    args:
    - "--context=dir:///workspace"
    - "--dockerfile=/workspace/Dockerfile"
    - "--destination=registry.io/myapp:${GIT_COMMIT}"
    - "--cache=true"
    - "--cache-repo=registry.io/myapp/cache"
    volumeMounts:
    - name: kaniko-secret
      mountPath: /kaniko/.docker
```

```
Option 4: Buildah / Podman
  Similar to Kaniko — rootless image builds
  OCI-compliant, daemonless
  Supported by Red Hat/OpenShift environments
```

Agent Template inheritance in Jenkins Kubernetes Plugin:

```
// Global podTemplate defined in Jenkins configuration (inheritable by
pipelines):
// Manage Jenkins → Configure System → Kubernetes → Pod Templates

// In Jenkinsfile — inherit from parent template:
pipeline {
    agent {
        kubernetes {
            inheritFrom 'maven-base'   // Inherits containers from parent
template
            yaml '''
spec:
  containers:
  - name: special-tool
    image: my-special-tool:latest
    command: ["cat"]
    tty: true
'''  // Only adds special-tool — inherits jnlp + maven from parent
        }
    }
}
```

🏭 Real-World Production Example

Shopify's Kubernetes Agent Evolution: Shopify migrated from 200 static agents (EC2 instances) to Kubernetes ephemeral pods. The static agents cost $45K/month running 24/7. With K8s agents on EC2 Spot Instances, agents only run during builds. Cost dropped to $8K/month — 82% reduction. Pod cold start latency (45 seconds) was addressed by pre-warming agent pods during business hours using Kubernetes cluster proportional autoscaler. The Maven cache PVC reduced build times from 8 minutes to 3 minutes by caching dependencies.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ What's the difference between JNLP and SSH agents? When would you use each?

> SSH agents: the Controller SSHs into the agent machine and starts the agent process — the Controller initiates the connection. Requires the agent machine to be network-reachable from the Controller on port 22 and have Java installed. Use when agents are in the same network and SSH is allowed. JNLP (Inbound) agents: the agent process initiates an outbound connection to the

> Controller on port 50000 (or WebSocket). Use when agents are behind NAT, in private subnets, or in cloud environments where the Controller can't initiate SSH — the agent can reach the Controller outbound even if not reachable inbound. Common for Kubernetes agents (pods connect out) and Windows agents (where SSH setup is more complex).

Q2: What are the security concerns with Docker agents and how do you mitigate them?

> Two main concerns: (1) Privileged containers (DinD) give the container root access to the Kubernetes node. Mitigation: use Kaniko or Buildah for image builds instead of DinD — they build images without a daemon or privilege escalation. (2) Docker socket mounting (DooD) gives the build container access to the host's Docker daemon, meaning it can see and potentially access other containers. Mitigation: use a Docker daemon socket proxy (docker-socket-proxy) that restricts which Docker API calls are allowed, or switch to Kaniko. Additional mitigations: run agent containers as non-root users, enforce PodSecurityPolicy/OPA Gatekeeper to prevent privileged pods, use separate node pools for build workloads.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Kubernetes agent pod eviction.** Kubernetes can evict pods for memory pressure, spot instance interruption, or node drain. If a build pod is evicted mid-build, the build fails with "agent went offline" — often with no clear error. Monitor for eviction events and configure PodDisruptionBudgets or use Spot Instance termination handlers.
- **Maven/NPM cache in K8s agents.** By default, each pod gets an empty workspace — no dependency cache. Without caching, a Java build downloads 500MB of dependencies on every build. Use PersistentVolumeClaims with `ReadWriteMany` (EFS/NFS) for shared caches, or BuildKit cache mounts.
- **Agent label mismatch cascade.** Typo in label in Jenkinsfile + no agent has that label = build queues forever. The Controller does NOT fail the build with "no such label" — it just waits. Always monitor queue depth and add alerts for builds queued > 10 minutes.
- **JNLP agent reconnection storm on Controller restart.** When you restart Jenkins Controller, all 200 pod agents try to reconnect simultaneously. This can OOM the Controller. Kubernetes plugin handles this with connection retries, but monitor Controller memory during planned restarts.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Architecture (2.1)	Agents are the "worker" tier of the architecture
Kubernetes Plugin (5.5)	Implements Kubernetes dynamic agents
Jenkins at Scale (2.8)	Agent pool management is core to scaling strategy
Pipeline agent directive (3.x)	Jenkinsfile `agent {}` block selects agent type
Workspace Management (2.5)	Workspace exists on agent; ephemeral agents lose workspace

---

Topic 2.5 — Jenkins Workspace: How It Works, Cleanup, Shared Workspaces

🟡 Intermediate | Operational Detail

📌 What It Is — In Simple Terms

The **Jenkins Workspace** is the directory on an agent machine where a build's working files live — source code checked out from Git, build outputs, test reports, downloaded dependencies. It's the build's private sandbox on the agent's filesystem.

Understanding workspaces matters because they're the source of several production problems: disk space exhaustion, stale file contamination between builds, and performance issues from large uncleaned workspaces.

⚙️ How It Works

Workspace Location and Naming

```
Default workspace location on agent:
  ${AGENT_WORKSPACE_ROOT}/${JOB_NAME}/

Example:
  Agent remoteFS: /home/jenkins/workspace
  Job name:       myapp/feature%2Fnew-checkout   (URL-encoded branch name)
  Workspace:      /home/jenkins/workspace/myapp/feature%2Fnew-checkout/

For concurrent builds of the same job:
  Build #45:  /home/jenkins/workspace/myapp@2/   (@ suffix for
concurrency)
  Build #46:  /home/jenkins/workspace/myapp@3/
```

Workspace Lifecycle

```
Build STARTS on agent:
  ↓
Does workspace already exist on this agent?
  YES → By default, reuse it (stale files may exist!)
  NO  → Create fresh workspace
  ↓
git checkout runs:
  → Files from previous build remain unless explicitly cleaned
  → Only changed files overwritten by git
  ↓
Build steps run (compile, test, etc.)
```

```
  ↓
Build ENDS:
  → Workspace NOT deleted by default
  → Files accumulate across builds
  → disk usage grows over time
```

The Stale Files Problem

```
Build #1: Generates target/report-v1.html
Build #2: report-v1.html was removed from code
Build #2 runs WITHOUT workspace cleanup:
  → target/report-v1.html STILL EXISTS in workspace
  → git checkout doesn't delete files not tracked anymore
  → Build #2 might accidentally reference old files
  → test results from #1 could contaminate #2 archives

This is the #1 source of "works on my machine but fails in CI" issues
that are actually "worked in build #45 but fails in #46"
```

⚙️ Workspace Cleanup Strategies

Strategy 1: `deleteDir()` — Clean Before Build

```
pipeline {
    agent { label 'linux' }
    stages {
        stage('Cleanup') {
            steps {
                // Delete entire workspace before build starts
                deleteDir()
            }
        }
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build') {
            steps { sh 'mvn clean package' }
        }
    }
}
```

Strategy 2: `cleanWs()` — Workspace Cleanup Plugin (Recommended)

```
pipeline {
    agent { label 'linux' }

    options {
        // Clean before EVERY build — most common production pattern
        skipDefaultCheckout(true)
    }

    stages {
        stage('Checkout') {
            steps {
                // Clean workspace before checkout
                cleanWs()
                checkout scm
            }
        }
    }

    post {
        // Or: clean AFTER build (keeps workspace for debugging failures)
        // Choose based on your needs:
        always {
            // cleanWs()  ← uncomment to clean after every build
        }
        success {
            cleanWs()   // Clean on success; keep on failure for debugging
        }
    }
}
```

Strategy 3: `git clean` — Fastest, Preserves Git History

```
stage('Checkout') {
    steps {
        checkout([
            $class: 'GitSCM',
            branches: [[name: env.BRANCH_NAME]],
            extensions: [
                // Clean before checkout — fastest approach
                // Uses git clean -fdx internally
                [$class: 'CleanBeforeCheckout',
deleteUntrackedNestedRepositories: true],
                // OR clean after checkout:
                [$class: 'CleanCheckout',
deleteUntrackedNestedRepositories: true]
            ],
            userRemoteConfigs: [[
                url: 'https://github.com/myorg/myapp.git',
                credentialsId: 'github-credentials'
            ]]
```

```
        ])
    }
}
```

Strategy 4: Agent-Level Workspace Cleanup (Workspaces Cleanup Plugin)

```yaml
// Configure in Jenkins → Manage Nodes → Agent → Workspace retention
// Or via JCasC:
jenkins:
  nodes:
  - permanent:
      name: "build-agent-1"
      # Auto-clean workspaces not used in 7 days:
      retentionStrategy:
        demand:
          idleDelay: 0
          inDemandDelay: 0
      # OR use Workspace Cleanup plugin periodic cleanup
```

Stash / Unstash — Passing Files Between Stages/Agents

```
// When stages run on DIFFERENT agents, they have DIFFERENT workspaces
// Use stash/unstash to pass files between them:

pipeline {
    agent none
    stages {
        stage('Build') {
            agent { label 'linux-1' }
            steps {
                sh 'mvn clean package'
                // Stash built artifacts — stored on Controller
temporarily
                stash name: 'built-jar',
                      includes: 'target/*.jar,target/surefire-reports/**'
                // Stash limit: 100MB default, configure with:
                // -
Dorg.jenkinsci.plugins.workflow.support.pickles.serialization.RiverWriter.
MAXIMUM_SIZE=200mb
            }
        }

        stage('Integration Test') {
            agent { label 'linux-2' }  // Different agent — no access to
linux-1's workspace
            steps {
                unstash 'built-jar'   // Retrieve stashed files from
Controller
```

```
                sh 'java -jar target/myapp.jar --integration-test'
            }
        }
    }
}
```

Shared Workspaces — When Multiple Stages Share Files

```
// Approach 1: Same agent for all stages (workspace naturally shared)
pipeline {
    agent { label 'linux' }  // All stages run on SAME agent
    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
                // target/ directory created in workspace
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'
                // Can access target/ from Build stage — same workspace ✅
            }
        }
    }
}

// Approach 2: Explicit stash/unstash for different agents
// See example above

// Approach 3: Shared volume (K8s agents)
// All containers in the same pod share emptyDir workspace:
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: maven
    image: maven:3.9
    command: ["cat"]
    tty: true
    volumeMounts:
    - name: workspace
      mountPath: /workspace
  - name: docker
    image: docker:24
    command: ["cat"]
    tty: true
    volumeMounts:
    - name: workspace     # Same volume — shared workspace
```

```
        mountPath: /workspace
    volumes:
    - name: workspace
      emptyDir: {}          # Shared between containers in same pod
  '''
        }
    }
    stages {
        stage('Build') {
            steps {
                container('maven') {
                    sh 'cd /workspace && mvn clean package'
                }
            }
        }
        stage('Docker Build') {
            steps {
                container('docker') {
                    // Can access /workspace/target/*.jar from maven stage
✅
                    sh 'docker build -t myapp:latest /workspace'
                }
            }
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Workspace	Agent-local directory for build working files
`deleteDir()`	Delete the current directory (entire workspace if called at root)
`cleanWs()`	Workspace Cleanup plugin — clean workspace with patterns and timing options
`CleanBeforeCheckout`	Git extension — `git clean -fdx` before checkout (fastest)
Stash/Unstash	Copy files from agent to Controller (stash) and back (unstash) — cross-agent file transfer
Workspace Locking	Prevent concurrent builds from sharing/corrupting workspace
Workspace Size Monitoring	Track disk usage per workspace — prevent disk exhaustion
`@tmp` **workspace**	Temporary workspace created by withDockerContainer — separate from main workspace

---

💬 Short Crisp Interview Answer

> *"The Jenkins workspace is the directory on an agent where build files live — git checkout, compiled artifacts, test reports. By default, workspaces persist between builds on the same agent, which can cause stale file contamination — files from previous builds polluting current ones. The recommended practice is to clean the workspace before each build using either `cleanWs()` from the Workspace Cleanup plugin or `CleanBeforeCheckout` git extension, which runs `git clean -fdx`. When stages run on different agents, they have isolated workspaces — files are passed between them using `stash` and `unstash`, which temporarily stores files on the Controller. For Kubernetes pod agents with multiple containers, containers in the same pod can share a workspace via an emptyDir volume mounted at the same path."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Workspace disk exhaustion.** Without cleanup, agent disks fill up with workspaces from hundreds of builds. On Kubernetes agents (ephemeral pods), this isn't an issue — the pod (and workspace) is destroyed after the build. On static agents, workspace accumulation is the #1 cause of "disk full" incidents. Monitor `/data/jenkins/workspace` disk usage.
- **`stash` has a 100MB default size limit.** Stashing large artifacts (fat JARs, Docker context directories) silently fails or is very slow. For large files, use an artifact repository (Nexus, S3) instead of stash.
- **Concurrent builds and workspace collisions.** If two builds of the same job run concurrently on the same agent, Jenkins appends `@2`, `@3` to workspace directory names. But if the job uses hardcoded file paths (`/home/jenkins/workspace/myapp/output.txt`) instead of `${WORKSPACE}` variable, builds collide. Always use `${WORKSPACE}` environment variable.
- **`deleteDir()` vs `cleanWs()` behavioral difference.** `deleteDir()` deletes the directory itself — the workspace directory is gone. `cleanWs()` cleans contents but preserves the directory. Using `deleteDir()` in `post { always {} }` means the workspace directory must be recreated on the next build, causing a brief extra overhead.
- **Git submodules and clean.** `CleanBeforeCheckout` doesn't clean git submodule directories by default. Set `deleteUntrackedNestedRepositories: true` to clean submodule directories as well.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Agent Types (2.4)	Workspace lives on agent; Kubernetes agents have ephemeral workspaces
Jenkins Home Directory (2.6)	JENKINS_HOME on Controller has its own workspace directory for built-in node
Stash/Unstash (3.10)	Mechanism for cross-agent file transfer
Parallel Stages (3.7)	Parallel stages on different agents need stash/unstash
Jenkins at Scale (2.8)	Disk management across many agents is a scaling concern

---

Topic 2.6 — Jenkins Home Directory Structure

🟡 Intermediate | Operational Anatomy

📌 What It Is — In Simple Terms

`JENKINS_HOME` is the root directory where Jenkins stores everything: job definitions, build history, plugin files, credentials, agent configuration, system configuration, and logs. Understanding this directory structure is essential for backup/restore, disaster recovery, performance tuning, and debugging.

Knowing JENKINS_HOME separates someone who's administered Jenkins from someone who's only used it.

⚙️ The Full Directory Structure

```
$JENKINS_HOME/                          ← Root (default: /var/jenkins_home
or /var/lib/jenkins)
│
├── config.xml                          ← Master Jenkins system
configuration (XML)
│                                         Security settings, auth, global
properties
│                                         DO NOT edit manually — use JCasC
instead
│
├── credentials.xml                     ← Encrypted credential store
│                                         Contains all credentials defined
at global scope
│                                         AES-256 encrypted with
jenkins.model.Jenkins.crumbSalt
│
├── secret.key                          ← Master encryption key for
credentials
│                                         CRITICAL: Back up this file
│                                         Without it, credentials.xml is
unreadable
│
├── secret.key.not-so-secret            ← Legacy key (older Jenkins
versions)
│
├── secrets/                            ← Secrets directory
│   ├── master.key                      ← AES master key (32 bytes)
│   ├── hudson.util.Secret              ← Encryption cipher for credential
values
│   ├── initialAdminPassword            ← One-time setup password (deleted
after setup)
│   └── org.jenkinsci.main.modules.cli.auth.Authentication ← CLI auth key
│
├── users/                              ← Local user accounts (if using
```

```
Jenkins user DB)
│   ├── admin_12345/
│   │   └── config.xml                  ← User profile, password hash, API
tokens
│   └── developer_67890/
│       └── config.xml
│
├── jobs/                               ← ⭐ ALL JOB DEFINITIONS LIVE HERE
│   ├── myapp-pipeline/
│   │   ├── config.xml                  ← Job configuration (SCM,
triggers, params)
│   │   ├── nextBuildNumber             ← Counter for next build number
│   │   └── builds/                     ← ALL BUILD HISTORY
│   │       ├── 45/                     ← Build #45 directory
│   │       │   ├── build.xml           ← Build metadata (result,
duration, timestamp)
│   │       │   ├── log                 ← Build console output (plain
text)
│   │       │   ├── archive/            ← Archived artifacts from this
build
│   │       │   │   └── target/myapp.jar
│   │       │   ├── workflow/           ← Pipeline CPS state (program.dat)
│   │       │   │   └── 2.xml           ← Serialized pipeline execution
state
│   │       │   └── changelog.xml       ← SCM changelog for this build
│   │       ├── 46/
│   │       └── ...
│   │
│   └── my-multibranch-job/            ← Multibranch Pipeline parent
│       ├── config.xml                  ← Multibranch config (branch
detection rules)
│       └── branches/                   ← Sub-jobs per branch
│           ├── main/
│           │   ├── config.xml
│           │   └── builds/
│           └── feature%2Fnew-checkout/  ← URL-encoded branch name
│               ├── config.xml
│               └── builds/
│
├── plugins/                            ← ⭐ ALL INSTALLED PLUGINS
│   ├── kubernetes/
│   │   ├── kubernetes.jpi              ← Plugin binary (JAR-based
archive)
│   │   └── kubernetes/                 ← Unpacked plugin directory
│   ├── git/
│   │   ├── git.jpi
│   │   └── git/
│   └── workflow-aggregator/
│       └── ...
│
├── nodes/                              ← AGENT CONFIGURATION
│   ├── build-agent-linux-1/
│   │   └── config.xml                  ← Agent settings (SSH, labels,
executors)
```

```
│   └── build-agent-docker-1/
│       └── config.xml
│
├── workspace/                          ← WORKSPACES FOR BUILT-IN NODE
│   │                                     (only used if Controller has
executors > 0)
│   └── (generally should be empty in production)
│
├── updates/                            ← Plugin update metadata cache
│   └── default.json                    ← Update center catalog
(downloaded from update center)
│
├── userContent/                        ← Static files served at
/userContent/ URL
│                                         Used for custom images, scripts
in job descriptions
│
├── logs/                               ← Custom log appenders (if
configured)
│
├── fingerprints/                       ← MD5 fingerprint database
│                                         Tracks which build produced
which artifact
│
├── casc_configs/                       ← JCasC configuration files (if
using JCasC plugin)
│   └── jenkins.yaml
│
└── .gitconfig, .ssh/                  ← Git/SSH config for the jenkins
user
    .m2/, .npm/                         ← Tool caches (Maven, npm) — grows
large!
```

📁 Critical Files for Operations

Backup Priority

```
CRITICAL (must back up — cannot recreate):
  $JENKINS_HOME/secrets/master.key
  $JENKINS_HOME/secrets/hudson.util.Secret
  → Without these, ALL stored credentials are permanently lost
  → Even if you have credentials.xml, it's encrypted with these keys

HIGH (should back up — hard to recreate):
  $JENKINS_HOME/config.xml                ← System configuration
  $JENKINS_HOME/credentials.xml           ← All credentials
  $JENKINS_HOME/jobs/*/config.xml         ← All job definitions
  $JENKINS_HOME/nodes/*/config.xml        ← Agent configurations
  $JENKINS_HOME/users/*/config.xml        ← User accounts
```

```
OPTIONAL (nice to have, can regenerate):
  $JENKINS_HOME/jobs/*/builds/            ← Build history (can be large)
  $JENKINS_HOME/plugins/*.jpi             ← Plugin binaries (re-
downloadable)
  $JENKINS_HOME/updates/                  ← Update catalog cache
```

Build Log Storage

```
# Each build's console log stored in: jobs/<name>/builds/<n>/log
# For 1000 builds with 1MB logs each: 1GB per job

# Check largest jobs:
du -sh $JENKINS_HOME/jobs/*/builds/ | sort -h | tail -20

# Build retention configuration (in Jenkinsfile):
pipeline {
    options {
        buildDiscarder(logRotator(
            numToKeepStr:     '20',   // Keep last 20 builds
            daysToKeepStr:    '30',   // Keep builds from last 30 days
            artifactDaysToKeepStr: '7',   // Keep artifacts 7 days
            artifactNumToKeepStr:  '5'    // Keep artifacts from last 5
builds
        ))
    }
}

# Or: configure globally in Jenkins → Manage Jenkins → Configure System
# → Default Build Retention Strategy
```

Plugin Update and Management

```
# Check installed plugin versions:
curl -s https://jenkins.example.com/pluginManager/api/json?depth=1 \
  -u admin:${TOKEN} | \
  python3 -c "import sys,json; [print(f\"{p['shortName']}:
{p['version']}\") \
    for p in json.load(sys.stdin)['plugins'] if p['active']]"

# Find outdated plugins:
curl -s https://jenkins.example.com/pluginManager/api/json?depth=1 \
  -u admin:${TOKEN} | \
  python3 -c "import sys,json; data=json.load(sys.stdin); \
    [print(f\"{p['shortName']}: {p['version']} → UPDATE AVAILABLE\") \
    for p in data['plugins'] if p.get('hasUpdate')]"

# The plugin directory structure:
ls $JENKINS_HOME/plugins/kubernetes/
```

```
# kubernetes.jpi  ← the plugin archive (Java archive)
# kubernetes/     ← unpacked plugin files (classes, resources)
#   WEB-INF/
#   META-INF/
#   index.jelly   ← plugin UI templates

# Disabled plugin:
# kubernetes.jpi.disabled  ← .disabled suffix = plugin installed but
inactive
```

🔑 Key Files Summary

File/Directory	Contains	Critical?
`config.xml`	Jenkins system configuration	High
`credentials.xml`	Encrypted credentials	Critical (with keys)
`secrets/master.key`	Root encryption key	**Critical**
`secrets/hudson.util.Secret`	Secondary encryption key	**Critical**
`jobs/*/config.xml`	Job definitions	High
`jobs/*/builds/`	Build history, logs, artifacts	Medium
`plugins/*.jpi`	Plugin binaries	Low (re-downloadable)
`nodes/*/config.xml`	Agent configurations	High
`users/*/config.xml`	User profiles and API tokens	High
`workspace/`	Built-in node workspaces	Low (should be empty)

💬 Short Crisp Interview Answer

> *"JENKINS_HOME is the root directory where Jenkins stores everything. The most critical files are in `secrets/` — `master.key` and `hudson.util.Secret` are the encryption keys for all stored credentials. Without them, your `credentials.xml` is permanently unreadable, even with a backup. Job definitions are in `jobs/<name>/config.xml`, and build history in `jobs/<name>/builds/<number>/`. Plugins are in `plugins/` as `.jpi` files. For backup, the absolute minimum is the `secrets/` directory and `jobs/*/config.xml` — credentials and job definitions. In production, I use a daily backup job that archives these to S3 or a secure NFS mount, and I test restores quarterly. Disk management is also critical — build logs in `builds/` can consume hundreds of GB without a retention policy."*

🔬 Deep Dive Answer

Credential Encryption Deep Dive:

```
# Jenkins credential encryption:
# 1. Master key stored in: secrets/master.key (random 32 bytes)
# 2. Secondary key: secrets/hudson.util.Secret (encrypted with master.key)
# 3. Each credential: AES-256 encrypted with hudson.util.Secret

# credentials.xml excerpt:
# <com.cloudbees.plugins.credentials.impl.UsernamePasswordCredentialsImpl>
#   <id>github-token</id>
#   <username>jenkins-bot</username>
#   <password>{AQAAABAAAAAwH7rW...}</password>  ← AES encrypted
#
</com.cloudbees.plugins.credentials.impl.UsernamePasswordCredentialsImpl>

# Decryption is only possible with matching secrets/ files
# This is why: you CANNOT restore credentials to a NEW Jenkins instance
#              by only copying credentials.xml — you must also copy
secrets/

# Security implication: if an attacker gets access to JENKINS_HOME,
# they have both the encrypted credentials AND the keys to decrypt them
# → JENKINS_HOME must be treated as a SECRET store — restrict filesystem
access
```

JENKINS_HOME Disk Usage Analysis:

```
# Identify disk hogs in JENKINS_HOME:
du -sh $JENKINS_HOME/jobs/* | sort -h | tail -20    # Largest jobs
du -sh $JENKINS_HOME/plugins/             # Plugin size (usually 1-3GB)
du -sh $JENKINS_HOME/jobs/*/builds/*/archive/  # Archived artifacts

# Automated cleanup with Jenkins API:
# Delete builds older than 30 days for a specific job:
JENKINS_URL=https://jenkins.example.com
JOB_NAME=myapp-pipeline
TOKEN=admin:${ADMIN_TOKEN}

# Get build numbers:
curl -s "${JENKINS_URL}/job/${JOB_NAME}/api/json?
tree=builds[number,timestamp]" \
  -u ${TOKEN} | \
  python3 -c "
import sys, json
from datetime import datetime, timedelta
data = json.load(sys.stdin)
cutoff = datetime.now() - timedelta(days=30)
for b in data['builds']:
    ts = datetime.fromtimestamp(b['timestamp']/1000)
    if ts < cutoff:
        print(b['number'])
" | while read num; do
    curl -X POST "${JENKINS_URL}/job/${JOB_NAME}/${num}/doDelete" -u
```

```
${TOKEN}
done
```

🏭 Real-World Production Example

GitLab's Jenkins Migration: When GitLab migrated their internal Jenkins, the team discovered they had accumulated 2TB of build artifacts in `JENKINS_HOME/jobs/*/builds/*/archive/`. The `master.key` and `secrets/` were fortunately backed up daily. The migration playbook: (1) Export job configs with `jobs/*/config.xml` (2) Copy `secrets/` directory (3) Install fresh Jenkins, copy secrets first (4) Import job configs via API (5) Re-download all plugins from update center (6) Rebuild all agent connections. Total migration: 4 hours. Without the `secrets/` backup, all 340 stored credentials would have required manual re-entry.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ What is the minimum you need to back up to fully restore Jenkins?

> The absolute minimum for a complete restore: (1) `$JENKINS_HOME/secrets/` — especially `master.key` and `hudson.util.Secret` — without these, all credentials are permanently lost, not just inaccessible. (2) `$JENKINS_HOME/config.xml` — system configuration. (3) `$JENKINS_HOME/jobs/*/config.xml` — all job definitions. (4) `$JENKINS_HOME/credentials.xml` — with the matching secrets files. (5) `$JENKINS_HOME/nodes/*/config.xml` — agent configurations. (6) `$JENKINS_HOME/users/` — if using local user DB. Plugins can be re-downloaded from the update center — they don't need to be backed up. Build history (`builds/`) is optional — losing it means you can't see historical build results, but Jenkins functions fully without it. The key insight: **always back up `secrets/` first and separately**.

⚠ Tricky Edge Cases & Gotchas

- **⚠ The `master.key` rotation trap.** If you restore credentials.xml from backup but the `secrets/` directory is from a different Jenkins instance (e.g., you copied credentials from prod to staging but forgot to copy secrets), ALL credentials will be unreadable. Always keep secrets/ and credentials.xml from the same Jenkins instance together.
- **Plugin `.disabled` files.** If you disable a plugin in Jenkins UI, it creates `plugin-name.jpi.disabled` in the plugins directory. If you then copy the plugins directory to a new Jenkins, the disabled plugin stays disabled — but many admins don't realize this, expecting all plugins in the directory to be active.
- **`nextBuildNumber` file.** If you delete builds manually from the filesystem (not via Jenkins API) and the `nextBuildNumber` file is not updated, the next build might try to create a directory that already exists or have a conflicting build number. Always delete builds via Jenkins API, not `rm -rf`.
- **JENKINS_HOME on NFS.** The `jobs/` directory is written to constantly during builds (build logs are streamed to disk in real time). NFS latency causes significant Jenkins slowdown. Use local block storage for JENKINS_HOME; if you must use network storage, ensure low-latency (<1ms) NFS or use a fast SAN.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Installation (2.2)	Installation populates JENKINS_HOME with initial config
Jenkins HA and DR (2.7)	Backup strategy is based on JENKINS_HOME knowledge
Pipeline Durability (4.7)	CPS state stored in `jobs/*/builds/*/workflow/`
Credentials Management (3.12)	Credentials live in JENKINS_HOME with encryption keys
JCasC Plugin (5.7)	`casc_configs/` directory in JENKINS_HOME

Topic 2.7 — Jenkins HA and Disaster Recovery ⚠

🔴 Advanced | Critical Production Knowledge

📌 What It Is — In Simple Terms

Jenkins High Availability (HA) addresses the fact that Jenkins has **no native HA solution** — there is no built-in active-active clustering, no automatic failover, and no hot standby. A single Jenkins Controller is a **Single Point of Failure (SPOF)**. If it goes down, all CI/CD stops.

Disaster Recovery (DR) is the plan for restoring Jenkins after an outage — hardware failure, data corruption, accidental deletion, or site disaster.

This is heavily tested because it reveals whether a candidate has operated Jenkins at production scale versus just used it.

🔍 The Core Problem

```
Jenkins Architecture Reality:
  ┌──────────────┐
  │  Controller  │  ← SPOF — one instance, no native clustering
  │  (Primary)   │
  └──────────────┘
         │
         │
    Builds queue here
    Config stored here
    Credentials here
    Plugins here
    Build history here

  If this machine dies:
    ✗ All in-flight builds → FAILED/LOST
    ✗ Build queue → LOST
```

```
    ✗ CI/CD system → DOWN
    ✗ Developers → blocked

  Time to recover: depends entirely on your DR plan
    No DR plan: hours to days
    Good DR plan: 15-30 minutes
    Great DR + warm standby: < 5 minutes
```

⚙️ HA and DR Strategies

Strategy 1: Kubernetes Pod with PVC (Modern Standard)

```yaml
# Jenkins on Kubernetes — the PVC provides durability:

# Jenkins deployment: always 1 replica
# If pod crashes → Kubernetes restarts it automatically on same/different
node
# JENKINS_HOME on PVC → survives pod restarts and node failures

apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins
  namespace: jenkins
spec:
  replicas: 1               # ← Always 1. NOT scalable to 2+ (no native
clustering)
  strategy:
    type: Recreate           # ← NOT RollingUpdate. Only one Jenkins pod
at a time
                             # RollingUpdate would briefly have 2
controllers
                             # → data corruption risk
  template:
    spec:
      containers:
      - name: jenkins
        image: jenkins/jenkins:lts-jdk17
        volumeMounts:
        - name: jenkins-home
          mountPath: /var/jenkins_home
      volumes:
      - name: jenkins-home
        persistentVolumeClaim:
          claimName: jenkins-pvc    # ← JENKINS_HOME on durable storage
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: jenkins-pvc
```

```
spec:
  accessModes:
  - ReadWriteOnce            # ← ReadWriteOnce: one pod at a time —
enforces single controller
  storageClassName: fast-ssd
  resources:
    requests:
      storage: 100Gi
```

```
Recovery flow on pod crash:
  Pod crashes → K8s detects (30-60s)
  → K8s schedules new pod
  → New pod mounts same PVC (JENKINS_HOME intact)
  → Jenkins starts, reads config from JENKINS_HOME
  → Agents reconnect
  → Jenkins back online: ~2-5 minutes

  In-flight builds: FAILED (must be manually re-triggered)
  Build history: INTACT (on PVC)
  Configuration: INTACT (on PVC)
```

Strategy 2: Regular Backup to S3/Object Storage

```
// Backup Jenkinsfile — runs as scheduled pipeline:
pipeline {
    agent { label 'linux' }

    triggers {
        cron('H 2 * * *')   // Daily at 2am (offset by H to reduce
thundering herd)
    }

    environment {
        BACKUP_BUCKET = 's3://company-jenkins-backups'
        JENKINS_HOME  = '/var/jenkins_home'
        TIMESTAMP     = sh(script: 'date +%Y%m%d-%H%M%S', returnStdout:
true).trim()
    }

    stages {
        stage('Backup Config and Credentials') {
            steps {
                script {
                    // CRITICAL: Backup secrets FIRST and SEPARATELY
                    sh """
                        # Archive secrets (most critical - encrypt before
uploading)
                        tar czf /tmp/jenkins-secrets-${TIMESTAMP}.tar.gz \
                            -C ${JENKINS_HOME} \
```

```
                            secrets/

                        # Encrypt with GPG before sending to S3
                        gpg --batch --yes \
                            --recipient jenkins-backup@example.com \
                            --encrypt /tmp/jenkins-
secrets-${TIMESTAMP}.tar.gz

                        # Upload encrypted secrets
                        aws s3 cp /tmp/jenkins-
secrets-${TIMESTAMP}.tar.gz.gpg \
                            ${BACKUP_BUCKET}/secrets/jenkins-
secrets-${TIMESTAMP}.tar.gz.gpg

                        # Clean up local encrypted file
                        shred -u /tmp/jenkins-secrets-${TIMESTAMP}.tar.gz
                        shred -u /tmp/jenkins-
secrets-${TIMESTAMP}.tar.gz.gpg
                    """

                    // Backup job configs and system config
                    sh """
                        tar czf /tmp/jenkins-config-${TIMESTAMP}.tar.gz \
                            -C ${JENKINS_HOME} \
                            config.xml \
                            credentials.xml \
                            jobs/*/config.xml \
                            nodes/*/config.xml \
                            users/ \
                            plugins/*.jpi \
                            casc_configs/

                        aws s3 cp /tmp/jenkins-config-${TIMESTAMP}.tar.gz
\
                            ${BACKUP_BUCKET}/config/jenkins-
config-${TIMESTAMP}.tar.gz

                        # Prune old backups: keep 30 days
                        aws s3 ls ${BACKUP_BUCKET}/config/ | \
                            awk '{print $4}' | \
                            sort | head -n -30 | \
                            xargs -I{} aws s3 rm
${BACKUP_BUCKET}/config/{}
                    """
                }
            }
        }

        stage('Backup Build History (Optional)') {
            steps {
                sh """
                    # Only backup last 7 days of builds (they get large)
                    find ${JENKINS_HOME}/jobs -name 'build.xml' \
                        -newer /tmp/last-build-backup \
```

```
                        -exec tar czf /tmp/jenkins-
builds-${TIMESTAMP}.tar.gz {} +

                    aws s3 cp /tmp/jenkins-builds-${TIMESTAMP}.tar.gz \
                        ${BACKUP_BUCKET}/builds/jenkins-
builds-${TIMESTAMP}.tar.gz

                    touch /tmp/last-build-backup
                """
            }
        }

        stage('Verify Backup') {
            steps {
                sh """
                    # Verify backup uploaded successfully
                    aws s3 ls ${BACKUP_BUCKET}/config/jenkins-
config-${TIMESTAMP}.tar.gz
                    aws s3 ls ${BACKUP_BUCKET}/secrets/jenkins-
secrets-${TIMESTAMP}.tar.gz.gpg

                    echo "✅ Backup verified: ${TIMESTAMP}"
                """
            }
        }
    }

    post {
        failure {
            slackSend channel: '#ops-alerts',
                message: "🚨 Jenkins backup FAILED: ${BUILD_URL}"
        }
    }
}
```

Strategy 3: Warm Standby Controller

```
Architecture: Active/Warm-Standby

Active Controller (PRIMARY):
  - Handles all builds and traffic
  - JENKINS_HOME on shared NFS/EFS/NFS

Warm Standby Controller (SECONDARY):
  - Runs but is DISABLED (not serving traffic)
  - Same JENKINS_HOME (shared storage) OR receives periodic rsync

Failover mechanism (MANUAL):
  1. Primary fails
  2. Ops team detects (monitoring alert)
  3. Update DNS/load balancer: jenkins.example.com → standby IP
```

```
    4. Start Jenkins on standby
    5. Standby reads same JENKINS_HOME → comes up with full state
    6. Agents reconnect to new IP

  RTO (Recovery Time Objective): ~10-15 minutes (manual failover)
  RPO (Recovery Point Objective): 0 (shared storage, no data loss)

  Note: Active-Active (two Controllers simultaneously) is NOT supported
  Both controllers CANNOT write to the same JENKINS_HOME simultaneously
  → Data corruption
```

Strategy 4: JCasC + GitOps = Fast Rebuild

```
  Philosophy: Don't try to backup and restore a monolithic JENKINS_HOME.
  Instead: make Jenkins fully reproducible from code in minutes.

  Requirements:
    1. All job definitions in Jenkinsfiles in app repos
       (Multi-branch pipelines auto-discover on startup)

    2. All system config in JCasC YAML in version control

    3. Plugin list in plugins.txt in version control

    4. No credentials stored in Jenkins GUI — use Vault or external secrets

  Rebuild playbook (when Jenkins is completely lost):
    Step 1: Deploy fresh Jenkins from Helm chart/Docker (5 min)
    Step 2: Jenkins starts, reads JCasC YAML → system configured (2 min)
    Step 3: Plugins installed from plugins.txt (5 min + restart)
    Step 4: Multi-branch scans run → all pipelines from Jenkinsfiles
  discovered (10 min)
    Step 5: Agents reconnect
    Total: ~25 minutes to fully operational Jenkins

    Drawback: Build history is LOST
    Mitigation: Export DORA metrics and key build results to external
  systems
                (Elasticsearch, Datadog) before relying on this strategy
```

Strategy 5: CloudBees CI (Enterprise HA)

```
  CloudBees CI (commercial Jenkins distribution) adds:
    - Operations Center: central management plane for multiple Controllers
    - High Availability: CloudBees-proprietary active/hot-standby
    - Client Controllers: isolated per-team Controllers under central
  governance
    - Cluster Operations: rolling upgrades, centralized plugin management
```

```
This is the enterprise answer to Jenkins HA — but requires commercial
license.
Open source Jenkins has NO native HA.

Interview context: Know that open-source Jenkins has no HA,
explain your mitigation strategies, and know that CloudBees CI exists as
the commercial solution.
```

🔑 Key Concepts

Concept	Meaning
SPOF	Single Point of Failure — Jenkins Controller is a SPOF by design
RTO	Recovery Time Objective — how long to restore service after failure
RPO	Recovery Point Objective — how much data loss is acceptable
Warm Standby	Secondary Controller ready to take over, not serving traffic
Cold Standby	Secondary Controller not running — must be started manually
PVC	Kubernetes PersistentVolumeClaim — durable storage for JENKINS_HOME
Recreate Strategy	Kubernetes deployment strategy ensuring only ONE Jenkins pod at a time
JCasC + GitOps DR	Rebuild Jenkins from code rather than restoring backup
ThinBackup	Popular Jenkins plugin for scheduled JENKINS_HOME backups
CloudBees CI	Commercial Jenkins distribution with native HA capabilities

---

💬 Short Crisp Interview Answer

> *"Jenkins has no native High Availability — the Controller is a Single Point of Failure. In production, I mitigate this through multiple layers: First, I run Jenkins on Kubernetes with JENKINS_HOME on a PersistentVolumeClaim using Recreate deployment strategy (never RollingUpdate, which could briefly create two concurrent Controllers and corrupt data). Kubernetes handles automatic pod restart on failures, typically restoring Jenkins in 2-5 minutes. Second, I implement daily backups of JENKINS_HOME — specifically the `secrets/` directory (encryption keys), `jobs/*/config.xml`, and `credentials.xml` — to encrypted S3 storage. Third, I use JCasC and Pipeline as Code so the entire Jenkins configuration is version-controlled and the system can be rebuilt from scratch in 25 minutes. For regulated environments needing hard HA SLAs, the answer is CloudBees CI, which provides a commercial active/hot-standby solution."*

🔬 Deep Dive Answer

The ReadWriteOnce vs ReadWriteMany PVC Gotcha:

```
ReadWriteOnce PVC: mounted by only ONE pod at a time
  → Correct for Jenkins — enforces single Controller
  → On pod migration: old pod must FULLY TERMINATE before new pod gets PVC
  → Migration latency: old pod termination timeout (default 30s) + new pod
startup

ReadWriteMany PVC (NFS/EFS): can be mounted by multiple pods
  → DANGEROUS for Jenkins — don't use this hoping it enables HA
  → Two Jenkins Controllers writing to same JENKINS_HOME simultaneously
    = database corruption, build history loss, credential file corruption
  → Jenkins is not designed for concurrent write access to JENKINS_HOME

Correct K8s HA pattern:
  1. ReadWriteOnce PVC on fast SSD storage
  2. Deployment with Recreate strategy (not RollingUpdate)
  3. Pod disruption budget: minAvailable: 0 (allows Jenkins to be down
during updates)
  4. Liveness + readiness probes to detect hangs
  5. Alert on pod restarts (PagerDuty rule: Jenkins pod restarted)
```

Monitoring for Proactive Failure Prevention:

```yaml
# Prometheus alerting rules for Jenkins:
groups:
- name: jenkins_alerts
  rules:

  # Alert if Jenkins is unreachable
  - alert: JenkinsDown
    expr: up{job="jenkins"} == 0
    for: 2m
    labels:
      severity: critical
    annotations:
      summary: "Jenkins Controller is DOWN"
      runbook: "https://wiki/jenkins-outage-runbook"

  # Alert on high heap usage (OOM risk)
  - alert: JenkinsHighHeap
    expr: |
      jvm_memory_used_bytes{job="jenkins", area="heap"} /
      jvm_memory_max_bytes{job="jenkins", area="heap"} > 0.85
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Jenkins heap usage > 85% — OOM risk"

  # Alert on build queue depth (agents may be down)
  - alert: JenkinsBuildQueueHigh
    expr: jenkins_queue_size_value > 20
```

```
      for: 10m
      labels:
        severity: warning
      annotations:
        summary: "Jenkins build queue > 20 — possible agent issues"

    # Alert on too many executor-less agents (all offline)
    - alert: JenkinsAllAgentsOffline
      expr: |
        sum(jenkins_executor_count_value) == 0
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "All Jenkins agents are offline"

    # Alert on disk usage
    - alert: JenkinsHomeDiskHigh
      expr: |
        (node_filesystem_size_bytes{mountpoint="/var/jenkins_home"} -
         node_filesystem_free_bytes{mountpoint="/var/jenkins_home"}) /
        node_filesystem_size_bytes{mountpoint="/var/jenkins_home"} > 0.80
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: "JENKINS_HOME disk > 80% — clean up builds/artifacts"
```

🏭 Real-World Production Example

Zalando's Jenkins DR Story: Zalando (European fashion e-commerce) had a Jenkins SPOF incident in 2018 — a botched plugin upgrade corrupted JENKINS_HOME. They didn't have a tested restore procedure. Recovery took 6 hours: manually re-creating ~180 job configs from memory/documentation, manually re-entering 40+ credentials, and rebuilding agent configurations.

Post-incident actions:

1. Migrated to JCasC — all config in version control
2. Implemented ThinBackup plugin with hourly backups to S3
3. Added "DR Test" to runbook — quarterly restore test from backup
4. Moved to Kubernetes with PVC — automated pod restart
5. Created "rebuild from scratch" playbook: 20 minutes total

Lesson: The backup is not the DR plan. A tested restore procedure is.

❓ Common Interview Questions & Strong Answers

Q1: ⚠ "Jenkins doesn't support HA — how do you handle this in production?"

> This is the key Jenkins HA question and the correct answer is: Jenkins has no native HA, but we mitigate the risk through: (1) Kubernetes deployment — if the Controller pod crashes, K8s restarts it in 2-5 minutes, with JENKINS_HOME on a PVC so no state is lost. (2) JCasC + Pipeline as Code — Jenkins can be rebuilt from scratch from code in 25 minutes if the PVC is lost. (3) Encrypted daily backups of JENKINS_HOME — especially `secrets/` directory — to S3. (4) Monitoring and alerting — PagerDuty on pod restarts, heap alerts before OOM. (5) Tested DR runbook — quarterly restore drills. If hard HA is a business requirement, the answer is CloudBees CI (commercial) or a different CI platform (GitHub Actions, GitLab CI) that has native HA/SaaS resilience.

Q2: How do you test your Jenkins DR plan?

> Quarterly DR drills: (1) Spin up a staging Jenkins instance from scratch — only allowed to use the backup files (no access to production Jenkins). (2) Apply JCasC YAML, install plugins from `plugins.txt`, configure agents. (3) Verify all pipelines are discovered via Multi-Branch scan. (4) Attempt to decrypt and use a stored credential — confirm `secrets/` backup is intact and decryptable. (5) Run a sample build to verify end-to-end. (6) Measure and document RTO (how long it actually took). If you've never tested your restore, you don't have a DR plan — you have a backup that might work.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Using `RollingUpdate` strategy in Kubernetes for Jenkins** — this will briefly create a second Jenkins pod before terminating the first. If both pods start accessing the same JENKINS_HOME PVC simultaneously, you get data corruption. Always use `Recreate` strategy for Jenkins.
- **The `secrets/` backup must be encrypted.** These keys can decrypt all your credentials. If `secrets/` is backed up to S3 in plaintext, anyone with S3 read access can decrypt all Jenkins credentials. Always encrypt with GPG, KMS, or Vault before storing.
- **Plugin version lock-in after backup restore.** If you backed up `plugins/*.jpi` and restore to a new Jenkins with a different version, plugin compatibility issues can arise. Consider tracking plugin versions in `plugins.txt` and re-downloading from update center instead of restoring binary `.jpi` files.
- **Credential restore with wrong secret keys.** If you restore `credentials.xml` but accidentally use a different `secrets/master.key` (from a different Jenkins), all credentials show as garbled text. The files must be from the same Jenkins instance at the same point in time.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Home Directory (2.6)	DR depends on knowing exactly what to back up in JENKINS_HOME
Jenkins Installation (2.2)	Kubernetes deployment method enables pod-restart HA
JCasC Plugin (5.7)	JCasC-based DR — rebuild from code rather than restore from backup
Jenkins at Scale (2.8)	At scale, HA becomes more critical — more teams depending on Jenkins

Connected Topic	How It Connects
Pipeline as Code (1.6)	Pipelines in SCM = one less thing to back up
Monitoring (9.4)	Proactive monitoring prevents outages that require DR

Topic 2.8 — Jenkins at Scale: Controller Bottlenecks, Agent Pools, Load Distribution

🔴 Advanced | Enterprise Operations

📌 What It Is — In Simple Terms

Running Jenkins for a **large engineering organization** (50+ teams, 500+ jobs, 1000+ daily builds) introduces scaling challenges that don't exist in small deployments. This topic covers: what breaks first as Jenkins scales, how to identify bottlenecks, architectural patterns for distributing load, and when Jenkins starts reaching its scaling limits.

🔍 Why This Matters

A Jenkins deployment that works fine for 5 teams with 20 jobs starts failing at 50 teams with 500 jobs:

- Controller becomes slow to respond
- Builds queue for 20+ minutes when agents are available
- Plugin operations block the scheduler
- JVM garbage collection pauses cause timeouts
- Agents reconnect continuously
- Build log storage exhausts disk

Understanding these bottlenecks and solutions is what interviewers probe at senior/staff level.

⚙️ Controller Bottlenecks — Identified and Solved

Bottleneck 1: JVM Heap and GC

```
Symptom: Jenkins UI becomes slow, builds take time to start,
         "Unable to schedule" errors, GC pause logs

Root cause: JVM heap too small or GC configuration suboptimal
Jenkins heap usage grows with:
  - Number of concurrent builds (each build's state in memory)
  - Number of plugins (each loads classes into heap)
  - Build log streaming (logs buffered in memory before disk write)
  - Serialized pipeline state (CPS continuations in memory)
```

```
Diagnosis:
  # JVM metrics via JMX:
  java -jar cmdline-jmxclient-0.10.3.jar - \
    localhost:1099 \
    java.lang:type=Memory HeapMemoryUsage

  # Or via Prometheus JMX exporter:
  heap_used / heap_max  →  if > 85%, increase -Xmx

Solution:
  # Minimum: 4GB heap for Jenkins with 50+ concurrent builds
  -Xms4g -Xmx4g   # Set equal to avoid heap resize GC
  -XX:+UseG1GC
  -XX:MaxGCPauseMillis=200
  -XX:G1HeapRegionSize=16m   # Larger regions for large heaps

  # Monitor GC log:
  -
Xlog:gc*:file=/var/log/jenkins/gc.log:time,uptime:filecount=5,filesize=20m
```

Bottleneck 2: Excessive SCM Polling

```
Symptom: Controller CPU spikes every 5 minutes,
         network usage spikes, GitHub rate limit errors

Root cause: 200 jobs all polling Git every 5 minutes
  200 jobs × 12 polls/hour = 2,400 git ls-remote calls/hour FROM
Controller

Solution: Migrate ALL jobs from polling to webhooks
  1. GitHub: Settings → Webhooks → Add webhook
     URL: https://jenkins.example.com/github-webhook/
     Content type: application/json
     Events: Push, Pull Request

  2. In Jenkinsfile:
     triggers {
         githubPush()       // Replace: pollSCM('H/5 * * * *')
     }

  Result: 0 polling CPU overhead, builds start in <2s of push

  # Audit polling jobs:
  curl -s https://jenkins.example.com/api/json?tree=jobs[name,triggers] \
    -u admin:${TOKEN} | \
    python3 -c "import sys,json; data=json.load(sys.stdin); \
      [print(j['name']) for j in data['jobs'] \
       if any('SCMTrigger' in str(t) for t in j.get('triggers',[]))]"
```

Bottleneck 3: Too Many Executors on Too Few Agents (Static Pool Underprovisioning)

```
Symptom: Long queue times even though jobs seem to complete fast
         "Waiting for next available executor" for 20+ minutes

Calculation for right-sizing agent pool:

Concurrent builds needed = peak_builds_per_hour /
builds_per_executor_per_hour
  Example:
    Peak: 100 builds/hour
    Average build duration: 10 minutes (6 builds/executor/hour)
    Executors needed: 100 / 6 = ~17 executors
    Agents needed (4 exec/agent): 17 / 4 = ~5 agents
    Add 20% headroom: 6 agents

# Monitor with:
# jenkins_executor_count_value (total executors)
# jenkins_executor_in_use_value (busy executors)
# Utilization = in_use / count  → if > 85% consistently, add agents
```

Bottleneck 4: Plugin Bloat and Background Tasks

```
Symptom: Controller periodically "freezes" for 10-30 seconds,
         builds start late, UI unresponsive at intervals

Root cause: Poorly written plugins run expensive operations on main thread
  - SCM polling (even with webhooks, some plugins still poll)
  - Plugin update checks (disable in air-gapped environments)
  - Fingerprint cleanup (background thread)
  - Workspace enumeration

Diagnosis:
  # Thread dump during freeze:
  curl https://jenkins.example.com/threadDump -u admin:${TOKEN}
  # Look for: threads BLOCKED in plugin code, not in remoting or Jetty

  # Disable unnecessary background work:
  JAVA_OPTS="...
    -Dhudson.model.DownloadService.never=true  # Disable update center
checks
    -Djenkins.fingerprints.FingerprintCleanupThread.disabled=true  # If
not using fingerprints
    -Dhudson.triggers.SafeTimerTask.loggingLevel=OFF"  # Reduce timer task
log noise
```

Bottleneck 5: Build Log Streaming

```
Symptom: High IO on Controller disk during many concurrent builds

Root cause: Every pipeline step streams log output to Controller
JENKINS_HOME
  With 50 concurrent builds each generating 1KB/s of logs:
  50 × 1KB/s = 50KB/s write to JENKINS_HOME disk (constant)
  Over time: large log files filling disk

Solutions:
  1. External log storage:
     # Fluentd/Filebeat sidecar reading Jenkins logs → Elasticsearch
     # pipeline-logfile plugin → stores logs on agents, not controller

  2. Log level reduction:
     # Reduce verbose plugin logging
     def logger = Logger.getLogger('org.jenkinsci.plugins.kubernetes')
     logger.setLevel(Level.WARNING)

  3. Build retention:
     options {
         buildDiscarder(logRotator(numToKeepStr: '10', daysToKeepStr:
'7'))
     }
```

⚙️ Agent Pool Architecture Patterns

Pattern 1: Label-Based Heterogeneous Pool

```
Pool: linux-general    (10 agents, 4 executors each = 40 slots)
  Labels: linux, docker, java11, java17, maven, node, python3
  Use: 90% of builds

Pool: windows-build    (3 agents, 2 executors each = 6 slots)
  Labels: windows, dotnet, powershell
  Use: .NET and Windows builds

Pool: macos-ios        (2 agents, 1 executor each = 2 slots)
  Labels: macos, xcode, ios, swift
  Use: iOS app builds (requires real Mac hardware)

Pool: gpu-ml           (2 agents, 1 executor each = 2 slots)
  Labels: gpu, cuda, ml-training
  Use: ML model training pipelines

Benefits:
  - Right-sized for each workload
  - Labels route jobs to appropriate pool
  - Failure of one pool doesn't affect others
  - Cost-efficient: iOS pool is small, most jobs don't need it
```

Pattern 2: Kubernetes Dynamic Agent Pool

```
# Kubernetes node pools for Jenkins agents:

# General compute pool — runs 90% of builds
- name: jenkins-general
  machineType: n2-standard-4   # 4 vCPU, 16GB RAM
  diskSizeGb: 100
  nodeCount: 0                  # Scale to zero
  autoscaling:
    minNodeCount: 0
    maxNodeCount: 20            # Max 20 nodes × 4 executors = 80
concurrent builds
  labels:
    workload: jenkins-build
  taints:
  - key: jenkins-build
    value: "true"
    effect: NoSchedule          # Only Jenkins pods scheduled here

# High-memory pool for integration tests with large data
- name: jenkins-high-mem
  machineType: n2-highmem-8    # 8 vCPU, 64GB RAM
  nodeCount: 0
  autoscaling:
    minNodeCount: 0
    maxNodeCount: 5
  labels:
    workload: jenkins-highmem
  taints:
  - key: jenkins-highmem
    value: "true"
    effect: NoSchedule

# Spot/preemptible pool — 70% cost savings for tolerant builds
- name: jenkins-spot
  machineType: n2-standard-4
  spot: true                   # Spot instances
  nodeCount: 0
  autoscaling:
    minNodeCount: 0
    maxNodeCount: 50           # Many cheap nodes
  labels:
    workload: jenkins-spot
```

```
// Jenkinsfile — routing to appropriate pool:

// Standard build → general pool
```

```
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  tolerations:
  - key: "jenkins-build"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
  nodeSelector:
    workload: jenkins-build
...'''
        }
    }
}

// Cost-sensitive build → spot pool
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  tolerations:
  - key: "jenkins-spot"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
  nodeSelector:
    workload: jenkins-spot
...'''
        }
    }
}
```

Pattern 3: Federated Jenkins — Multiple Controllers

```
When a single Jenkins Controller can't handle the load:
  (Rule of thumb: > 500 concurrent builds, > 1000 jobs, > 100 agents)

Federated Architecture:

  Organization Jenkins (Operations Center — CloudBees) OR
  Internal Jenkins Router

  ┌──────────────────────────────────────────────────────────────────┐
  │                    FRONTEND ROUTING                              │
  │    jenkins.example.com → routes by team/org                      │
  └──────────────┬──────────────────────────┬──────────────────────┘
                 │                          │
  ┌──────────────▼──────────┐      ┌──────────────▼──────────┐
```

```
    │  CONTROLLER A      │       │   CONTROLLER B      │
    │  Team: Platform    │       │   Team: Product      │
    │  Jobs: 200         │       │   Jobs: 300          │
    │  Agents: 40        │       │   Agents: 60         │
    └────────────────────┘       └──────────────────────┘

  Benefits:
    - Each team/org has isolated Jenkins
    - Controller failure affects only one team
    - Independent upgrade cycles
    - Security isolation between teams

  Challenges:
    - Multiple systems to operate
    - No shared credential store (unless Vault)
    - Duplicate plugin management
    - Higher infrastructure cost
```

Pattern 4: Agent Pre-Warming (Kubernetes)

```
  Problem: Cold start latency for K8s agents (45-90 seconds)
    Developer pushes code → webhook → Jenkins schedules build
    → 60 seconds waiting for pod to start
    → Developers frustrated by perceived CI "slowness"

  Solution: Pre-warm agent pods during business hours

  # kubernetes-agent-prewarmer CronJob:
  apiVersion: batch/v1
  kind: CronJob
  metadata:
    name: jenkins-agent-prewarmer
  spec:
    # Keep agents warm during business hours: 7am-8pm Mon-Fri
    schedule: "0 7 * * 1-5"   # Start pre-warming at 7am
    jobTemplate:
      spec:
        template:
          spec:
            containers:
            - name: prewarmer
              image: bitnami/kubectl:latest
              command:
              - /bin/sh
              - -c
              - |
                # Scale jenkins-agent-pool node pool to minimum 5 nodes
                gcloud container clusters resize ${CLUSTER} \
                  --node-pool jenkins-general \
                  --num-nodes 5 \
                  --zone ${ZONE}
```

```
              echo "Pre-warmed 5 agent nodes"
---
# Scale down at night (8pm):
apiVersion: batch/v1
kind: CronJob
metadata:
  name: jenkins-agent-scaledown
spec:
  schedule: "0 20 * * 1-5"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: scaledown
            image: bitnami/kubectl:latest
            command: ["/bin/sh", "-c", "gcloud container clusters resize
... --num-nodes 0"]
```

🔑 Key Concepts

Concept	Meaning
Throughput	Number of builds Jenkins can complete per hour
Latency	Time from trigger to build start (queue wait + agent startup)
Controller Ceiling	Maximum builds/hour before Controller becomes the bottleneck (typically 200-500/hr)
Agent Pool	Group of homogeneous agents serving a specific workload
Horizontal Scaling	Adding more agents — the right way to scale Jenkins capacity
Vertical Scaling	Adding more CPU/RAM to Controller — limited benefit, wrong approach
Federated Jenkins	Multiple Controllers distributing load across teams/organizations
Cluster Proportional Autoscaler	K8s tool that scales agent node pools based on pending pod count
Build Sharding	Splitting a large test suite across multiple parallel agents
Throttle Plugin	Prevents any single job type from consuming all executors

---

💬 Short Crisp Interview Answer

> *"Jenkins scales horizontally by adding agents — never vertically by adding more executors to the Controller. The Controller is the scheduling brain, not the execution engine. Common bottlenecks at scale: JVM heap exhaustion from too many concurrent builds — solve with proper G1GC tuning and Xmx sizing; SCM polling load — replace all polling with webhooks; Jetty thread pool exhaustion from too many concurrent pipeline steps — solve with PERFORMANCE_OPTIMIZED durability. For agent*

> *pool architecture, I use label-based pools with Kubernetes dynamic agents on spot instances for cost efficiency, with pre-warming during business hours to reduce cold-start latency. When a single Controller can't scale further (typically beyond 200-300 concurrent builds), the answer is federated Jenkins — multiple Controllers split by team or function, with a shared Vault for credentials and a shared agent cluster."*

🔬 Deep Dive Answer

The 500-Concurrent-Build Rule of Thumb:

```
Jenkins Controller can handle approximately:
  - 200-500 concurrent builds (depending on JVM tuning and pipeline
complexity)
  - Before: scheduler latency increases, UI becomes sluggish

Factors that reduce this limit:
  - Complex declarative pipelines (more CPS serialization overhead)
  - Many plugins (more background threads)
  - High durability hints (more disk IO)
  - Large log output per build (more log streaming overhead)

Factors that increase this limit:
  - G1GC with large heap (reduces GC pauses)
  - PERFORMANCE_OPTIMIZED durability (reduces per-step Controller
overhead)
  - Webhooks instead of polling (eliminates polling thread overhead)
  - External log storage (reduces disk IO pressure on Controller)
  - Agents close to Controller (lower remoting latency)

Measurement:
  Monitor: jenkins_executor_in_use_value vs controller CPU and response
time
  When controller response > 2s at N concurrent builds → you've hit the
ceiling
  Solution: either optimize (durability, GC) or federate
```

Shared Library Caching at Scale:

```
// At scale, every pipeline loads Shared Library classes
// Without caching: each build downloads library from Git → Controller IO
// With caching: library compiled once per version, cached in memory

// Force Shared Library cache:
@Library('jenkins-shared-library@v2.4.0') _
// Pin to exact version → Jenkins can cache compiled classes
// vs:
@Library('jenkins-shared-library@main') _
// Floating branch → Jenkins must re-evaluate on every build
// → Higher SCM calls, higher compilation overhead at scale
```

Build Optimization for Throughput:

```
// Techniques to increase Jenkins throughput:

// 1. Parallelize within builds
stage('Tests') {
    parallel {
        stage('Unit')        { steps { sh 'pytest tests/unit/' } }
        stage('Integration') { steps { sh 'pytest tests/integration/' } }
        stage('Lint')        { steps { sh 'flake8 src/' } }
    }
}

// 2. Reduce checkouts (large repos slow down builds)
checkout([$class: 'GitSCM',
    extensions: [
        [$class: 'CloneOption', shallow: true, depth: 1],  // Shallow
clone
        [$class: 'SparseCheckoutPaths',                    // Only
checkout needed dirs
            sparseCheckoutPaths: [[path: 'src/'], [path: 'tests/']]]
    ],
    ...
])

// 3. Skip unchanged stages
stage('Build Docker') {
    when {
        anyOf {
            changeset 'Dockerfile'
            changeset 'src/**'
        }
    }
    steps { sh 'docker build ...' }
}

// 4. Reduce artifact archiving
// Don't archive everything — only what's needed for downstream jobs
archiveArtifacts artifacts: 'target/*.jar',
                 excludes:  'target/test-classes/**,target/site/**',
                 onlyIfSuccessful: true
```

🏭 Real-World Production Example

LinkedIn's Jenkins at Scale: LinkedIn operates Jenkins for 3,000+ engineers, 5,000+ jobs, and 15,000+ builds per day. Their architecture:

- 8 Jenkins Controllers, split by product area (Mobile, Backend, Data, Platform, etc.)

- Each Controller handles ~2,000 builds/day peak
- 2,000+ Kubernetes ephemeral agents on AWS EC2 Spot Instances
- Shared Vault for cross-team credentials
- All agents have Maven/npm caches via EFS PVCs
- Build queue depth monitored with PagerDuty alerts at queue > 50 items
- Controller heap sized at 8GB with G1GC — JVM metrics exported to Datadog
- All SCM triggers via webhooks — zero polling across all 5,000 jobs

Lessons from their scale:

1. The Controller-per-team model (federated) solved their scaling wall at 500 concurrent builds/Controller
2. Spot instances reduced agent costs by 68%
3. Maven cache PVCs reduced average build time from 8min to 2.5min
4. Webhook migration eliminated 40% of Controller CPU usage

❓ Common Interview Questions & Strong Answers

Q1: At what point does a single Jenkins Controller stop being sufficient, and what do you do?

> A single Controller starts showing stress signs around 200-300 concurrent builds: scheduler latency increases (builds start late despite available agents), the UI response time degrades to > 2 seconds, and GC pause logs show frequent collection. The right diagnosis is JVM metrics and thread dumps — not just "it feels slow." Optimization steps: first, tune G1GC and increase heap; second, migrate polling to webhooks; third, switch pipelines to PERFORMANCE_OPTIMIZED durability. If optimization doesn't solve it, federate: split teams across multiple Controllers. The split criteria should be organizational — teams operate independently, reducing the blast radius of any one Controller's issues.

Q2: How do you size an agent pool for a team with unpredictable build load?

> Kubernetes dynamic agents with autoscaling. I analyze historical build data: peak concurrent builds in the last 30 days, average and p99 build duration, and build frequency by time of day. From this, I calculate the steady-state and peak agent requirements. With K8s, I configure: minimum 2 always-on nodes (warm agents, 30s response to trigger), autoscaling up to N nodes where N covers the peak + 20% headroom. For cost, I use Spot/Preemptible instances for the autoscaling range — builds are interruptible (the build fails and can be retried). For the 2 always-on nodes, I use on-demand instances. This gives fast response time for normal load and elastic burst capacity at fraction of the cost.

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Vertical scaling the Controller doesn't scale Jenkins.** Adding 32 CPUs and 64GB RAM to the Controller JVM helps up to a point, but the fundamental bottleneck is single-threaded components in Jenkins (the scheduler, some plugin operations). Beyond ~8GB heap and ~8 CPUs, the returns diminish. Horizontal scaling (more Controllers or more agents) is the right answer.

- **Shared Library compilation at scale.** If 500 builds/hour all import the same Shared Library, Jenkins compiles the library Groovy scripts 500 times/hour. With library versioning (pinned tags), Jenkins caches compiled library classes — drastically reducing overhead. Unversioned libraries (floating `main` branch) defeat caching.
- **Kubernetes Cluster Autoscaler and Jenkins.** The K8s Cluster Autoscaler scales down nodes when they've been underutilized for 10 minutes (default). If a build pod lands on a node that then gets scaled down mid-build, the pod is evicted. Use `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` annotation on build pods to prevent mid-build eviction.
- **Agent connection limits on Controller.** The Jenkins remoting TCP port (50000) and the connection handling threads have limits. With 500+ concurrent agents, connection management overhead becomes significant. Use WebSocket agents (port 443) instead of TCP 50000 in large deployments — it's more firewall-friendly and has better connection management.

🔗 Connections to Other Topics

Connected Topic	How It Connects
Jenkins Architecture (2.1)	Architecture determines scaling pattern — horizontal (agents) not vertical (Controller)
Controller Internals (2.3)	Bottlenecks manifest in queue, scheduler, thread pool
Agent Types (2.4)	K8s dynamic agents are the scaling mechanism
Jenkins HA (2.7)	Scale increases failure impact — HA becomes more critical
Pipeline Observability (9.4)	Metrics identify which bottleneck you're hitting
Kubernetes Integration (6.3)	K8s is the agent scaling platform

---

📊 Category 2 Summary — Quick Reference

Topic	Core Concept	Interview Priority
2.1 Architecture	Controller orchestrates, Agents execute, Controller executors = 0	⭐⭐⭐⭐⭐
2.2 Installation	JCasC makes Jenkins reproducible; JVM tuning prevents OOM	⭐⭐⭐
2.3 Controller Internals	Build Queue states; CPS engine; Jetty thread pool exhaustion	⭐⭐⭐⭐
2.4 Agent Types	Static vs K8s dynamic; DinD vs Kaniko; JNLP vs SSH	⭐⭐⭐⭐⭐

Topic	Core Concept	Interview Priority
2.5 Workspaces	Clean before build; stash/unstash; Kubernetes shared volumes	⭐⭐⭐
2.6 Home Directory	`secrets/` = critical; backup minimum; disk management	⭐⭐⭐⭐
2.7 HA and DR	No native HA — Kubernetes PVC + JCasC + backups + tested restore	⭐⭐⭐⭐⭐ ⚠
2.8 Jenkins at Scale	Webhooks > polling; federated Controllers; K8s agents on spot	⭐⭐⭐⭐

🔑 The Mental Model for Jenkins Architecture

```
Think of Jenkins like a manufacturing plant:

CONTROLLER = Factory Management Office
  - Receives orders (build triggers via webhooks)
  - Decides which production line handles each order (agent scheduling)
  - Tracks inventory (build history, artifacts)
  - Manages suppliers (plugin ecosystem)
  - Never touches the machines (0 executors on Controller)

AGENTS = Production Lines
  - Actually do the work (build steps)
  - Specialized by capability (labels = java11, docker, windows)
  - Scale out by adding more lines (add agents)
  - Clean shop floor before each job (cleanWs)

EXECUTORS = Workstations per Production Line
  - One workstation = one concurrent job
  - 4 workstations × 5 lines = 20 parallel jobs

JENKINS_HOME = Company Records Room
  - Stores everything: job configs, build history, credentials
  - Fire-proof safe: secrets/ (encryption keys) — back up first
  - Clean it or it fills up: build retention policies

HA REALITY = The factory has one manager with no deputy
  - Manager gets sick → factory stops
  - Mitigation: keep manager healthy (monitoring)
  - Contingency: deputy trained and ready (warm standby/JCasC)
  - Backup: factory blueprints archived (JCasC + Pipeline as Code)
```

🧪 Self-Quiz — 10 Interview Questions to Answer Before Moving On

1. Why must the Jenkins Controller have 0 executors? What are the security and stability reasons?

2. Walk through what happens internally when a webhook fires and a build eventually starts on a Kubernetes agent.
3. What are the three build queue states? What does each mean and how do you fix a build stuck in each?
4. Explain the difference between SSH and JNLP agent connection methods. When would you use each?
5. A build fails with "agent went offline." What are the three most likely causes and how do you diagnose each?
6. What is the minimum you need to back up in JENKINS_HOME for a complete restore? Why is `secrets/` so critical?
7. Jenkins has no native HA. How would you architect a production Jenkins deployment to minimize downtime risk?
8. Your Jenkins Controller becomes slow and unresponsive during peak build hours. Walk through your diagnostic and resolution approach.
9. What is CPS transformation in Jenkins pipelines and why does it matter for pipeline development?
10. A team says their builds are queuing for 45 minutes despite available Kubernetes nodes. Walk through your investigation.

Next: Category 3 — Jenkins Pipelines Core (the most interview-tested category) Or: Say "quiz me" to test yourself on Category 2

> **Document Version:** Category 2 Complete | DevOps/SRE Interview Prep Series **Coverage:** 8 topics | Beginner → Advanced | Jenkins Architecture & Internals **Key theme:** Know Jenkins from the inside out — not just as a user, but as an operator